arXiv:2204.07137v1 [cs.LG] 14 Apr 2022

ACCELERATED POLICY LEARNING WITH PARALLEL DIFFERENTIABLE SIMULATION

Jie Xu[1,2], **Viktor Makoviychuk**[1], **Yashraj Narang**[1], **Fabio Ramos**[1,3],
Wojciech Matusik[2], **Animesh Garg**[1,4], **Miles Macklin**[1]

[1]NVIDIA [2]Massachusetts Institute of Technology [3]University of Sydney [4]University of Toronto

ABSTRACT

Deep reinforcement learning can generate complex control policies, but requires large amounts of training data to work effectively. Recent work has attempted to address this issue by leveraging differentiable simulators. However, inherent problems such as local minima and exploding/vanishing numerical gradients prevent these methods from being generally applied to control tasks with complex contact-rich dynamics, such as humanoid locomotion in classical RL benchmarks. In this work we present a high-performance differentiable simulator and a new policy learning algorithm (SHAC) that can effectively leverage simulation gradients, even in the presence of non-smoothness. Our learning algorithm alleviates problems with local minima through a smooth critic function, avoids vanishing/exploding gradients through a truncated learning window, and allows many physical environments to be run in parallel. We evaluate our method on classical RL control tasks, and show substantial improvements in sample efficiency and wall-clock time over state-of-the-art RL and differentiable simulation-based algorithms. In addition, we demonstrate the scalability of our method by applying it to the challenging high-dimensional problem of muscle-actuated locomotion with a large action space, achieving a greater than 17× reduction in training time over the best-performing established RL algorithm. More visual results are provided at: https://short-horizon-actor-critic.github.io/.

1 INTRODUCTION

Learning control policies is an important task in robotics and computer animation. Among various policy learning techniques, reinforcement learning (RL) has been a particularly successful tool to learn policies for systems ranging from robots (*e.g.,* Cheetah, Shadow Hand) (Hwangbo et al., 2019; Andrychowicz et al., 2020) to complex animation characters (*e.g.,* muscle-actuated humanoids) (Lee et al., 2019) using only high-level reward definitions. Despite this success, RL requires large amounts of training data to approximate the policy gradient, making learning expensive and time-consuming, especially for high-dimensional problems (Figure 1, Right). The recent development of differentiable simulators opens up new possibilities for accelerating the learning and optimization of control policies. A differentiable simulator may provide accurate first-order gradients of the task performance reward with respect to the control inputs. Such additional information potentially allows the use of efficient gradient-based methods to optimize policies. As recently Freeman et al. (2021) show, however, despite the availability of differentiable simulators, it has not yet been convincingly demonstrated that they can effectively accelerate policy learning in complex high-dimensional and contact-rich tasks, such as some traditional RL benchmarks. There are several reasons for this:

1. Local minima may cause gradient-based optimization methods to stall.
2. Numerical gradients may vanish/explode along the backward path for long trajectories.
3. Discontinuous optimization landscapes can occur during policy failures/early termination.

Because of these challenges, previous work has been limited to the optimization of open-loop control policies with short task horizons (Hu et al., 2019; Huang et al., 2021), or the optimization of policies for relatively simple tasks (*e.g.,* contact-free environments) (Mora et al., 2021; Du et al., 2021). In this work, we explore the question: *Can differentiable simulation accelerate policy learning in tasks with continuous closed-loop control and complex contact-rich dynamics?*

[cs.LG] 14 Apr 2022

并行可微仿真加速策略学习

Jie Xu[1,2], **Viktor Makoviychuk**[1], **Yashraj Narang**[1],**Fabio Ramos**[1,3],
Wojciech Matusik[2], **Animesh Garg**[1,4],**Miles Macklin**[1]

[1]英伟达 [2]麻省理工学院 [3]悉尼大学 [4]多伦多大学

摘要

深度强化学习能够生成复杂的控制策略，但需要大量训练数据才能有效工作。最近的研究试图通过利用可微仿真器来解决这一问题。然而，诸如局部极小值和梯度爆炸/消失等固有问题，阻碍了这些方法普遍应用于具有复杂富接触动力学的控制任务，例如经典强化学习基准中的人形机器人 locomotion。在本工作中，我们提出了一种高性能可微仿真器和一种新的策略学习算法（SHAC），该算法即使在存在非光滑性的情况下，也能有效利用仿真梯度。我们的学习算法通过平滑 critic 函数缓解了局部极小值问题，通过截断学习窗口避免了梯度消失/爆炸，并允许并行运行多个物理环境。我们在经典强化学习控制任务上评估了我们的方法，结果表明，与最先进的强化学习和基于可微仿真的算法相比，我们的方法在样本效率和挂钟时间上均有显著提升。此外，我们通过将该方法应用于具有高维动作空间的肌肉驱动 locomotion 这一挑战性高维问题，展示了其可扩展性，实现了比表现最佳的现有强化学习算法超过 17× 的训练时间减少。更多可视化结果请访问：https://short-horizon-actor-critic.github.io/。

1 引言

学习控制策略是机器人学和计算机动画中的一项重要任务。在各种策略学习技术中，强化学习（RL）已成为一种特别成功的工具，能够仅通过高层奖励定义，为从机器人（例如，Cheetah、Shadow Hand）（Hwangbo 等人，2019；Andrychowicz 等人，2020）到复杂动画角色（例如，肌肉驱动人形机器人）（Lee et al., 2019）等各类系统学习策略。尽管取得了这一成功，但 RL 需要大量训练数据来近似策略梯度，使得学习过程昂贵且耗时，尤其是对于高维问题（图 1，右）。最近，可微仿真器的发展为加速控制策略的学习和优化开辟了新的可能性。可微模拟器可提供任务性能奖励相对于控制输入的精确一阶梯度。此类额外信息 potentially 允许使用高效的基于梯度的方法来优化策略。然而，正如 Freeman 等人（2021）最近所示，尽管已有可微模拟器，但尚未令人信服地证明它们能有效加速复杂高维且富含接触的任务（如某些传统强化学习基准）中的策略学习。其原因有以下几点：

1. 局部极小值可能导致基于梯度的优化方法停滞。
2. 对于长轨迹，数值梯度可能在反向路径上消失或爆炸。
3. 不连续优化景观可能发生在策略失败或提前终止期间。

由于这些挑战，以往的工作仅限于优化具有短任务时域的开环控制策略（Hu et al., 2019；Huang et al., 2021），或优化相对简单任务的策略（例如，无接触环境）（Mora et al., 2021；Du et al., 2021）。在本工作中，我们探讨以下问题：可微仿真能否加速具有连续闭环控制和复杂富接触动力学的任务中的策略学习？

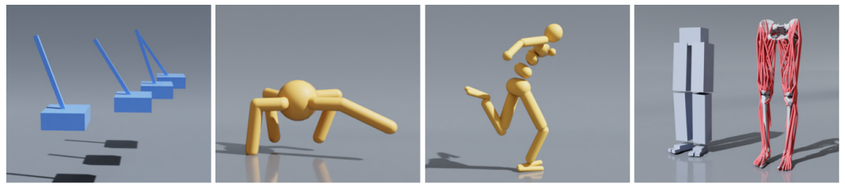

Figure 1: **Environments**: Here are some of our environments for evaluation. Three classical physical control RL benchmarks of increasing difficulty, from left: Cartpole Swing Up + Balance, Ant, and Humanoid. In addition, we train the policy for the high-dimensional muscle-tendon driven Humanoid MTU model from Lee et al. (2019). Whereas model-free reinforcement learning (PPO, SAC) needs many samples for such high-dimensional control problems, SHAC scales efficiently through the use of analytic gradients from differentiable simulation with a parallelized implementation, both in sample complexity and wall-clock time.

Inspired by actor-critic RL algorithms (Konda & Tsitsiklis, 2000), we propose an approach to effectively leverage differentiable simulators for policy learning. We alleviate the problem of local minima by using a critic network that acts as a smooth surrogate to approximate the underlying noisy reward landscape resulted by complex dynamics and occurrences of policy failures (Figure 2). In addition, we use a truncated learning window to shorten the backpropagation path to address problems with vanishing/exploding gradients, reduce memory requirements, and improve learning efficiency.

A further challenge with differentiable simulators is that the backward pass typically introduces some computational overhead compared to optimized forward-dynamics physics engines. To ensure meaningful comparisons, we must ensure that our learning method not only improves sample-efficiency, but also wall-clock time. GPU-based physics simulation has shown remarkable effectiveness for accelerating model-free RL algorithms (Liang et al., 2018; Allshire et al., 2021), given this, we develop a GPU-based *differentiable* simulator that can compute gradients of standard robotics models over many environments in parallel. Our PyTorch-based simulator allows us to connect high-quality simulation with existing algorithms and tools.

To the best of our knowledge, this work is the first to provide a fair and comprehensive comparison between gradient-based and RL-based policy learning methods, where fairness is defined as (a) benchmarking on both RL-favored tasks and differentiable-simulation-favored tasks, (b) testing complex tasks (*i.e.*, contact-rich tasks with long task horizons), (c) comparing to the state-of-the-art implementation of RL algorithms, and (d) comparing both sample efficiency and wall-clock time. We evaluate our method on standard RL benchmark tasks, as well as a high-dimensional character control task with over 150 actuated degrees of freedom (some of tasks are shown in Figure 1). We refer to our method as Short-Horizon Actor-Critic (SHAC), and our experiments show that SHAC outperforms state-of-the-art policy learning methods in both sample-efficiency and wall-clock time.

2 Related Work

Differentiable Simulation Physics-based simulation has been widely used in the robotics field (Todorov et al., 2012; Coumans & Bai, 2016). More recently, there has been interest in the construction of differentiable simulators, which directly compute the gradients of simulation outputs with respect to actions and initial conditions. These simulators may be based on auto-differentiation frameworks (Griewank & Walther, 2003; Heiden et al., 2021; Freeman et al., 2021) or analytic gradient calculation (Carpentier & Mansard, 2018; Geilinger et al., 2020; Werling et al., 2021). One challenge for differentiable simulation is the non-smoothness of contact dynamics, leading many works to focus on how to efficiently differentiate through linear complementarity (LCP) models of contact (Degrave et al., 2019; de Avila Belbute-Peres et al., 2018; Werling et al., 2021) or leverage a smooth penalty-based contact formulation (Geilinger et al., 2020; Xu et al., 2021).

Deep Reinforcement Learning Deep reinforcement learning has become a prevalent tool for learning control policies for systems ranging from robots (Hwangbo et al., 2019; OpenAI et al., 2019; Xu et al., 2019; Lee et al., 2020; Andrychowicz et al., 2020; Chen et al., 2021a), to complex animation characters (Peng et al., 2018; 2021; Liu & Hodgins, 2018; Lee et al., 2019). Model-free RL algorithms treat the underlying dynamics as a black box in the policy learning process. Among them, on-policy RL approaches (Schulman et al., 2015; 2017) improve the policy from the experience generated by the current policy, while off-policy methods (Lillicrap et al., 2016; Mnih et al.,

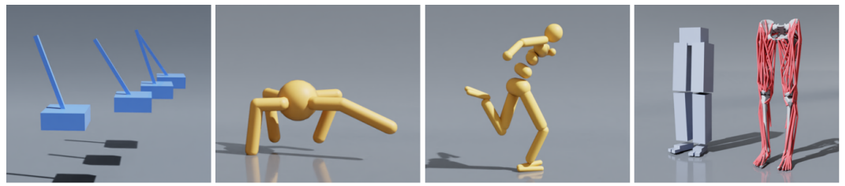

图 1：**环境**：以下是我们用于评估的部分环境。从左至右依次为三个难度递增的经典物理控制强化学习基准：CartPole Swing Up + Balance、Ant 和 Humanoid。此外，我们还针对 Lee et al. (2019) 提出的高维肌肉 - 肌腱驱动 Humanoid MTU 模型训练了策略。对于此类高维控制问题，无模型强化学习（如 PPO、SAC）需要大量样本，而 SHAC 则通过利用可微仿真的解析梯度并采用并行化实现，在样本复杂度和挂钟时间上均实现了高效扩展。

受演员 - 评论家强化学习算法（Konda 和 Tsitsiklis（2000））的启发，我们提出了一种方法，以有效利用可微仿真器进行策略学习。我们通过使用一个评论家网络来缓解局部极小值问题，该网络作为平滑代理，用于近似由复杂动力学和策略失败发生所导致的底层噪声奖励景观（图 2）。此外，我们使用截断学习窗口来缩短反向传播路径，从而解决梯度消失/爆炸问题，降低内存需求，并提高学习效率。

可微仿真器面临的另一项挑战是，与优化的前向动力学物理引擎相比，反向传播通常会引入一定的计算开销。为确保有意义的比较，我们必须保证我们的学习方法不仅能提高样本效率，还能缩短挂钟时间。鉴于基于 GPU 的物理仿真在加速无模型强化学习算法方面已展现出显著成效（Liang 等人（2018）；Allshire 等人（2021）），我们开发了一种基于 GPU 的可微模拟器，能够并行计算多个环境中标准机器人学模型的梯度。我们的基于 PyTorch 的模拟器使我们能够将高质量仿真与现有算法及工具无缝连接。

据我们所知，本文首次对基于梯度的策略学习方法与基于强化学习的策略学习方法进行了公平且全面的比较，其中公平性定义为：(a) 在有利于强化学习的任务和有利于可微仿真的任务上均进行基准测试；(b) 测试复杂任务（即，具有长任务时间范围的富含接触的任务）；(c) 与强化学习算法的最先进实现进行比较；以及 (d) 同时比较样本效率和挂钟时间。我们在标准强化学习基准任务以及一个拥有超过 150 个驱动自由度的高维角色控制任务上评估了我们的方法（部分任务如图 1 所示）。我们将我们的方法称为短视界演员 - 评论家（SHAC），实验结果表明，SHAC 在样本效率和挂钟时间方面均优于最先进的策略学习方法。

2 相关工作

可微仿真基于物理的仿真已广泛应用于机器人学领域（Todorov 等人（2012）；Coumans 和 Bai（2016））。最近，人们开始关注可微仿真器的构建，这类仿真器能够直接计算仿真输出相对于动作和初始条件的梯度。这些仿真器可能基于自动微分框架（Griewank 和 Walther（2003）；Heiden 等人（2021）；Freeman 等人（2021）），也可能基于解析梯度计算（Carpentier 和 Mansard（2018）；Geilinger 等人（2020）；Werling 等人（2021））。可微仿真面临的一个挑战是接触动力学的非光滑性，这导致许多研究致力于如何高效地对接触的线性互补问题（LCP）模型进行微分（Degrave 等人（2019）；de Avila Belbute-Peres 等人（2018）；Werling 等人（2021）），或利用平滑基于惩罚的接触公式（Geilinger 等人（2020）；Xu 等人（2021））。

深度强化学习深度强化学习已成为一种普遍工具，用于学习从机器人（Hwangbo 等人，2019；OpenAI 等，2019；Xu 等，2019；Lee 等人，2020；Andrychowicz 等人，2020；Chen 等，2021a）到复杂动画角色（Peng 等，2018；2021；Liu & Hodgins，2018；Lee et al., 2019）等各种系统的控制策略。无模型强化学习算法在策略学习过程中将底层动力学视为黑箱。其中，同策略强化学习方法（Schulman 等，2015；2017）利用当前策略生成的经验来改进策略，而异策略方法（Lillicrap 等，2016；Mnih 等，

2016; Fujimoto et al., 2018; Haarnoja et al., 2018) leverage all the past experience as a learning resource to improve sample efficiency. On the other side, model-based RL methods (Kurutach et al., 2018; Janner et al., 2019) have been proposed to learn an approximated dynamics model from little experience and then fully exploit the learned dynamics model during policy learning. The underlying idea of our method shares similarity with some prior model-based methods (Hafner et al., 2019; Clavera et al., 2020) that both learn in an actor-critic mode and leverage the gradient of the simulator "model". However, we show that our method can achieve considerably better wall-clock time efficiency and policy performance using parallel differentiable simulation in place of a learned model. A detailed note on comparison with model-based RL is presented in Appendix A.4.6.

Differentiable Simulation based Policy Learning The recent development of differentiable simulators enables the optimization of control policies via the provided gradient information. Backpropagation Through Time (BPTT) (Mozer, 1995) has been widely used in previous work to showcase differentiable systems (Hu et al., 2019; 2020; Liang et al., 2019; Huang et al., 2021; Du et al., 2021). However, the noisy optimization landscape and exploding/vanishing gradients in long-horizon tasks make such straightforward first-order methods ineffective. A few methods have been proposed to resolve this issue. Qiao et al. (2021) present a sample enhancement method to increase RL sample-efficiency for the simple MuJoCo Ant environment. However, as the method follows a model-based learning framework, it is significantly slower than state-of-the-art on-policy methods such as PPO (Schulman et al., 2017). Mora et al. (2021) propose interleaving a trajectory optimization stage and an imitation learning stage to detach the policy from the computation graph in order to alleviate the exploding gradient problem. They demonstrate their methods on simple control tasks (*e.g.,* stopping a pendulum). However, gradients flowing back through long trajectories of states can still create challenging optimization landscapes for more complex tasks. Furthermore, both methods require the full simulation Jacobian, which is not commonly or efficiently available in reverse-mode differentiable simulators. In contrast, our method relies only on first-order gradients. Therefore, it can naturally leverage simulators and frameworks that can provide this information.

3 METHOD

3.1 GPU-BASED DIFFERENTIABLE DYNAMICS SIMULATION

Conceptually, we treat the simulator as an abstract function $\mathbf{s}_{t+1} = \mathcal{F}(\mathbf{s}_t, \mathbf{a}_t)$ that takes a state $\mathbf{s}$ from a time $t \rightarrow t+1$, where $\mathbf{a}$ is a vector of actuation controls applied during that time-step (may represent joint torques, or muscle contraction signals depending on the problem). Given a differentiable scalar loss function $\mathcal{L}$, and its adjoint $\mathcal{L}^* = \frac{\partial \mathcal{L}}{\partial \mathbf{s}_{t+1}}$, the simulator backward pass computes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{s}_t} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}_{t+1}}\right)\left(\frac{\partial \mathcal{F}}{\partial \mathbf{s}_t}\right), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}_{t+1}}\right)\left(\frac{\partial \mathcal{F}}{\partial \mathbf{a}_t}\right) \tag{1}$$

Concatenating these steps allows us to propagate gradients through an entire trajectory.

To model the dynamics function $\mathcal{F}$, our physics simulator solves the forward dynamics equations

$$\mathbf{M}\ddot{\mathbf{q}} = \mathbf{J}^T\mathbf{f}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}), \tag{2}$$

where $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ are joint coordinates, velocities and accelerations respectively, $\mathbf{f}$ represents external forces, $\mathbf{c}$ includes Coriolis forces, and $\boldsymbol{\tau}$ represents joint-space actuation. The mass matrix $\mathbf{M}$, and Jacobian $\mathbf{J}$, are computed in parallel using one thread per-environment. We use the composite rigid body algorithm (CRBA) to compute articulation dynamics which allows us to cache the resulting matrix factorization at each step (obtained using parallel Cholesky decomposition) for re-use in the backwards pass. After determining joint accelerations $\ddot{\mathbf{q}}$ we perform a semi-implicit Euler integration step to obtain the updated system state $\mathbf{s} = (\mathbf{q}, \dot{\mathbf{q}})$.

For simple environments we actuate our agents using torque-based control, in which the policy outputs $\boldsymbol{\tau}$ at each time-step. For the more complex case of muscle-actuation, each muscle consists of a list of attachment sites that may pass through multiple rigid bodies, and the policy outputs activation values for each muscle. A muscle activation signal generates purely contractive forces (mimicking biological muscle) with maximum force prescribed in advance (Lee et al., 2019).

Analytic articulated dynamics simulation can be non-smooth and even discontinuous when contact and joint limits are introduced, and special care must be taken to ensure smooth dynamics. To model contact, we use the frictional contact model from Geilinger et al. (2020), which approximates Coulomb friction with a linear step function, and incorporate the contact damping force formulation

2016；Fujimoto 等，2018；Haarnoja 等，2018）则利用所有过往经验作为学习资源，以提高样本效率。另一方面，基于模型的强化学习方法（Kurutach 等，2018；Janner 等，2019）被提出用于从少量经验中学习近似动力学模型，然后在策略学习过程中充分利用该学到的动力学模型。我们方法的基本思想与一些先前的基于模型的方法（Hafner 等，2019；Clavera 等，2020）有相似之处，它们都采用演员 - 评论家模式进行学习，并利用模拟器“模型”的梯度。然而，我们表明，通过使用并行可微分仿真替代学习到的模型，我们的方法可以实现显著更好的挂钟时间效率和策略性能。关于与基于模型的强化学习的详细比较说明见附录 A.4.6。

基于可微仿真的策略学习可微模拟器的最新发展使得能够通过所提供的梯度信息来优化控制策略。时间反向传播（BPTT）(Mozer, 1995) 已被先前的工作广泛用于展示可微系统（Hu et al., 2019; 2020; Liang 等，2019; Huang et al., 2021; Du et al., 2021）。然而，在长时程任务中，噪声优化景观和梯度爆炸/消失问题使得此类直接的一阶梯度方法效果不佳。已有少数方法被提出以解决这一问题。Qiao 等（2021）提出了一种样本增强方法，以提高简单 MuJoCo 蚂蚁环境中的强化学习样本效率。然而，由于该方法遵循基于模型的学习框架，其速度显著慢于诸如 PPO（Schulman 等，2017）等最先进的 on-policy 方法。Mora 等（2021）提出交替进行轨迹优化阶段和模仿学习阶段，从而将策略从计算图中分离出来，以缓解梯度爆炸问题。他们在简单的控制任务（例如，停止摆锤）上展示了他们的方法。然而，对于更复杂的任务，沿状态长轨迹反向传播的梯度仍可能产生具有挑战性的优化景观。此外，这两种方法都需要完整的仿真雅可比矩阵，而这在逆向模式可微模拟器中通常无法高效获得。相比之下，我们的方法仅依赖于一阶梯度。因此，它可以自然地利用能够提供此类信息的模拟器和框架。

3 方法

3.1 基于 GPU 的可微分动力学仿真

从概念上讲，我们将模拟器视为一个抽象函数 $\mathbf{s}_{t+1} = \mathcal{F}(\mathbf{s}_t, \mathbf{a}_t)$，该函数接收时间 $t \rightarrow t+1$ 的状态 $\mathbf{s}$，其中 $\mathbf{a}$ 是在该时间步内施加的驱动控制向量（根据具体问题，可能代表关节扭矩或肌肉收缩信号）。给定一个可微分的标量损失函数 $\mathcal{L}$ 及其伴随 $\mathcal{L}^* = \frac{\partial \mathcal{L}}{\partial \mathbf{s}_{t+1}}$，模拟器的反向传播计算如下：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{s}_t} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}_{t+1}}\right)\left(\frac{\partial \mathcal{F}}{\partial \mathbf{s}_t}\right), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}_{t+1}}\right)\left(\frac{\partial \mathcal{F}}{\partial \mathbf{a}_t}\right) \tag{1}$$

将这些步骤连接起来，使我们能够沿整个轨迹传播梯度。

为了对动力学函数 $\mathcal{F}$ 进行建模，我们的物理模拟器求解前向动力学方程

$$\mathbf{M}\ddot{\mathbf{q}} = \mathbf{J}^T\mathbf{f}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}), \tag{2}$$

其中 $\mathbf{q}$ $\dot{\mathbf{q}}$ $\ddot{\mathbf{q}}$ 分别表示关节坐标、速度和加速度，$\mathbf{f}$ 代表外力，$\mathbf{c}$ 包含科里奥利力，$\boldsymbol{\tau}$ 代表关节空间驱动。质量矩阵 $\mathbf{M}$ 和雅可比矩阵 $\mathbf{J}$ 采用每个环境一个线程的方式并行计算。我们使用复合刚体算法（CRBA）来计算关节动力学，这使得我们能够在每一步缓存所得的矩阵分解结果（通过并行 Cholesky 分解获得），以便在反向传播中重复使用。在确定关节加速度 $\ddot{\mathbf{q}}$ 后，我们执行半隐式欧拉积分步骤以获得更新的系统状态 $\mathbf{s} = (\mathbf{q}, \dot{\mathbf{q}})$。

对于简单环境，我们使用基于力矩的控制来驱动智能体，其中策略在每个时间步输出 $\boldsymbol{\tau}$。对于更复杂的肌肉驱动情况，每块肌肉由一系列附着点组成，这些附着点可能穿过多个刚体，而策略则为每块肌肉输出激活值。肌肉激活信号会产生纯粹的收缩力（模拟生物肌肉），其最大力预先设定（Lee et al., 2019）。

引入接触和关节限位后，解析关节动力学仿真可能会变得不平滑甚至不连续，因此必须特别注意以确保平滑动力学。为了对接触进行建模，我们采用了 Geilinger 等 (2020) 提出的摩擦接触模型，该模型使用线性阶跃函数来近似库仑摩擦，并融合了接触阻尼力公式

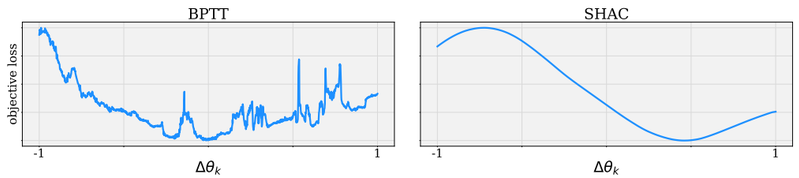

Figure 2: **Landscape comparison between BPTT and SHAC.** We select one single weight from a policy and change its value by $\Delta\theta_k \in [-1, 1]$ to plot the task loss landscapes of BPTT and SHAC w.r.t. one policy parameter. The task horizon is $H = 1000$ for BPTT, and the short horizon length for our method is $h = 32$. As we can see, longer optimization horizons lead to noisy loss landscape that are difficult to optimize, and the landscape of our method can be regarded as a smooth approximation of the real landscape.

from Xu et al. (2021) to provide better smoothness of the non-interpenetration contact dynamics:

$$\mathbf{f}_c = (-k_n + k_d\dot{d})\min(d, 0)\mathbf{n}, \quad \mathbf{f}_t = -\frac{\mathbf{v}_t}{\|\mathbf{v}_t\|}\min(k_t\|\mathbf{v}_t\|, \mu\|\mathbf{f}_c\|) \tag{3}$$

where $\mathbf{f}_c, \mathbf{f}_t$ are the contact normal force and contact friction force respectively, d and $\dot{d}$ are the contact penetration depth and its time derivative (negative for penetration), $\mathbf{n}$ is the contact normal direction, $\mathbf{v}_t$ is the relative speed at the contact point along the contact tangential direction, and k_n, k_d, k_t, μ are contact stiffness, contact damping coefficient, friction stiffness and coefficient of friction respectively.

To model joint limits, a continuous penalty-based force is applied:

$$\mathbf{f}_{\text{limit}} = \begin{cases} k_{\text{limit}}(q_{\text{lower}} - q), & q < q_{\text{lower}} \\ k_{\text{limit}}(q_{\text{upper}} - q), & q > q_{\text{upper}} \end{cases} \tag{4}$$

where k_{limit} is the joint limit stiffness, and $[q_{\text{lower}}, q_{\text{upper}}]$ is the bound for the joint angle q.

We build our differentiable simulator on PyTorch (Paszke et al., 2019) and use a source-code transformation approach to generate forward and backward versions of our simulation kernels (Griewank & Walther, 2003; Hu et al., 2020). We parallelize the simulator over environments using distributed GPU kernels for the dense matrix routines and evaluation of contact and joint forces.

3.2 Optimization Landscape Analysis

Although smoothed physical models improve the local optimization landscape, the combination of forward dynamics and the neural network control policy renders each simulation step non-linear and non-convex. This problem is exacerbated when thousands of simulation steps are concatenated and the actions in each step are coupled by a feedback control policy. The complexity of the resulting reward landscape leads simple gradient-based methods to easily become trapped in local optima.

Furthermore, to handle agent failure (*e.g.,* a humanoid falling down) and improve sample efficiency, early termination techniques are widely used in policy learning algorithms (Brockman et al., 2016). Although these have proven effective for model-free algorithms, early termination introduces additional discontinuities to the optimization problem, which makes methods based on analytical gradients less successful.

To analyze this problem, inspired by previous work (Parmas et al., 2018), we plot the optimization landscape in Figure 2 (Left) for a humanoid locomotion problem with a 1000-step task horizon. Specifically, we take a trained policy, perturb the value of a single parameter θ_k in the neural network, and evaluate performance for the policy variations. As shown in the figure, with long task horizons and early termination, the landscape of the humanoid problem is highly non-convex and discontinuous. In addition, the norm of the gradient $\frac{\partial \mathcal{L}}{\partial \theta}$ computed from backpropagation is larger than 10^6. We provide more landscape analysis in Appendix A.5. Thus, most previous works based on differentiable simulation focus on short-horizon tasks with contact-free dynamics and no early termination, where pure gradient-based optimization (*e.g.*, BPTT) can work successfully.

3.3 Short-Horizon Actor-Critic (SHAC)

To resolve the aforementioned issues of gradient-based policy learning, we propose the Short-Horizon Actor-Critic method (SHAC). Our method concurrently learns a policy network (*i.e.*, actor) π_θ and a value network (*i.e.*, critic) V_ϕ during task execution, and splits the entire task horizon into

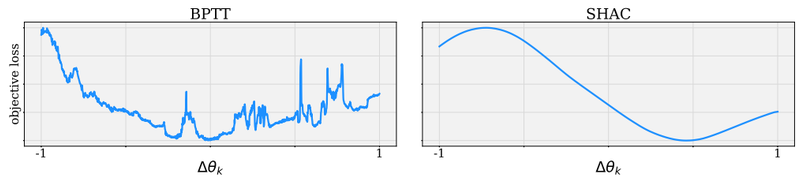

图 2：**BPTT 与 SHAC 的景观对比**。我们从策略中选取单个权重，将其值改变 $\Delta\theta_k \in [-1, 1]$，以绘制 BPTT 和 SHAC 关于单一策略参数的任务损失景观。BPTT 的任务时间范围为 $H = 1000$，而我们方法的短视界长度为 $h = 32$。由此可见，更长的优化时间范围会导致噪声较大的损失景观，从而难以优化；而我们方法的景观可视为真实景观的平滑近似。

（源自 Xu 等 (2021)），以提供更具平滑性的非穿透接触动力学：

$$\mathbf{f}_c = (-k_n + k_d\dot{d})\min(d, 0)\mathbf{n}, \quad \mathbf{f}_t = -\frac{\mathbf{v}_t}{\|\mathbf{v}_t\|}\min(k_t\|\mathbf{v}_t\|, \mu\|\mathbf{f}_c\|) \tag{3}$$

其中$\mathbf{f}_c$，$\mathbf{f}_t$ 分别为接触法向力和接触摩擦力，d 和 $\dot{d}$ 分别为接触穿透深度及其时间导数（穿透时为负值），$\mathbf{n}$ 为接触法向方向，$\mathbf{v}_t$ 为接触点沿接触切向方向的相对速度，而k_n，k_d，k_t，μ 分别为接触刚度、接触阻尼系数、摩擦刚度和摩擦系数。

为了对关节限位进行建模，施加了基于连续惩罚的力：

$$\mathbf{f}_{\text{limit}} = \begin{cases} k_{\text{limit}}(q_{\text{lower}} - q), & q < q_{\text{lower}} \\ k_{\text{limit}}(q_{\text{upper}} - q), & q > q_{\text{upper}} \end{cases} \tag{4}$$

其中 k_{limit} 是关节限位刚度，$[q_{\text{lower}}, q_{\text{upper}}]$ 是关节角度 q 的边界。

我们在 PyTorch（Paszke 等，2019）基础上构建了可微模拟器，并采用源代码转换方法生成模拟内核的前向和反向版本（Griewank 和 Walther（2003）；Hu 等，2020）。我们利用分布式 GPU 内核执行稠密矩阵例程以及接触力和关节力的评估，从而在多个环境上并行化该模拟器。

3.2 优化景观分析

尽管平滑物理模型改善了局部优化景观，但前向动力学与神经网络控制策略的结合使得每个仿真步都呈现非线性和非凸性。当数千个仿真步被串联起来，且每一步的动作通过反馈控制策略相互耦合时，这一问题会进一步加剧。由此产生的奖励景观的复杂性导致简单的基于梯度的方法极易陷入局部最优。

此外，为了处理智能体失败（例如，人形机器人摔倒）并提高样本效率，提前终止技术在策略学习算法中被广泛使用（Brockman 等，2016）。尽管这些方法对无模型算法已被证明有效，但提前终止为优化问题引入了额外的不连续性，这使得基于解析梯度的方法难以取得成功。

为了分析这一问题，受先前工作（Parmas 等，2018）的启发，我们在图 2（左）中绘制了一个任务时间范围为 1000 步的人形机器人运动问题的优化景观。具体而言，我们选取一个训练策略，扰动神经网络中单个参数 θ_k 的值，并评估策略变体的性能。如图所示，在长任务时间范围和提前终止的条件下，人形机器人问题的优化景观高度非凸且不连续。此外，由反向传播计算得到的梯度 $\frac{\partial \mathcal{L}}{\partial \theta}$ 的范数大于 10^6。我们在附录 A.5 中提供了更多的景观分析。因此，大多数基于可微仿真的先前工作都专注于短 horizon 任务、无接触动力学且无提前终止的场景，在这些场景中，纯基于梯度的优化（例如，BPTT）可以成功运行。

3.3 短视界演员 - 评论家（SHAC）

为解决上述基于梯度的策略学习问题，我们提出了短时域演员 - 评论家方法（SHAC）。我们的方法在任务执行过程中并发学习策略网络（即，演员）π_θ 和价值网络（即，critic）V_ϕ，并将整个任务时间范围划分为

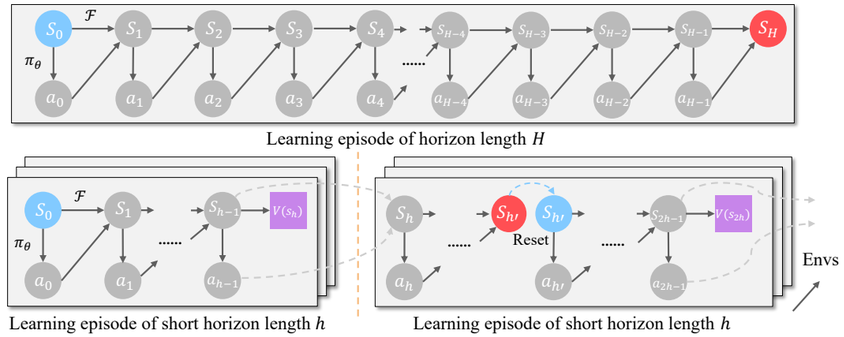

Figure 3: **Computation graph of BPTT and SHAC**. **Top**: BPTT propagates gradients through an entire trajectory in each learning episode. This leads to noisy loss landscapes, increased memory, and numerical gradient problems. **Bottom**: SHAC subdivides the trajectory into short optimization windows across learning episodes. This results in a smoother surrogate reward function and reduces memory requirements, enabling parallel sampling of many trajectories. The environment is reset upon early termination happens. Solid arrows denote gradient-preserving computations; dashed arrows denote locations at which the gradients are cut off.

several sub-windows of smaller horizons across learning episodes (Figure 3). A multi-step reward in the sub-window plus a terminal value estimation from the learned critic is used to improve the policy network. The differentiable simulation is used to backpropagate the gradient through the states and actions inside the sub-windows to provide an accurate policy gradient. The trajectory rollouts are then collected and used to learn the critic network in each learning episode.

Specifically, we model each of our control problems as a finite-horizon Markov decision process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, reward function $\mathcal{R}$, transition function $\mathcal{F}$, initial state distribution $\mathcal{D}_{\mathbf{s}_0}$, and task horizon H. At each step, an action vector $\mathbf{a}_t$ is computed by a feedback policy $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$. While our method does not constrain the policy to be deterministic or stochastic, we use the stochastic policy in our experiments to facilitate extra exploration. Specifically, the action is sampled by $\mathbf{a}_t \sim \mathcal{N}(\mu_\theta(\mathbf{s}_t), \sigma_\theta(\mathbf{s}_t))$. The transition function $\mathcal{F}$ is modeled by our differentiable simulation (Section 3.1). A single-step reward $r_t = \mathcal{R}(\mathbf{s}_t, \mathbf{a}_t)$ is received at each step. The goal of the problem is to find the policy parameters θ that maximize the expected finite-horizon reward.

Our method works in an on-policy mode as follows. In each learning episode, the algorithm samples N trajectories $\{\tau_i\}$ of short horizon $h \ll H$ in parallel from the simulation, which continue from the end states of the trajectories in the previous learning episode. The following policy loss is then computed:

$$\mathcal{L}_\theta = -\frac{1}{Nh}\sum_{i=1}^{N}\left[\left(\sum_{t=t_0}^{t_0+h-1}\gamma^{t-t_0}\mathcal{R}(\mathbf{s}_t^i, \mathbf{a}_t^i)\right) + \gamma^h V_\phi(\mathbf{s}_{t_0+h}^i)\right], \tag{5}$$

where $\mathbf{s}_t^i$ and $\mathbf{a}_t^i$ are the state and action at step t of the i-th trajectory, and $\gamma < 1$ is a discount factor introduced to stabilize the training. Special handling such as resetting the discount ratio is conducted when task termination happens during trajectory sampling.

To compute the gradient of the policy loss $\frac{\partial \mathcal{L}_\theta}{\partial \theta}$, we treat the simulator as a differentiable layer (with backward pass shown in Eq. 1) in the PyTorch computation graph and perform regular backpropagation. We apply reparameterization sampling method to compute the gradient for the stochastic policy. For details of gradient computation, see Appendix A.1. Our algorithm then updates the policy using one step of Adam (Kingma & Ba, 2014). The differentiable simulator plays a critical role here, as it allows us to fully utilize the underlying dynamics linking states and actions, as well as optimize the policy, producing better short-horizon reward inside the trajectory and a more promising terminal state for the sake of long-term performance. We note that the gradients are cut off between learning episodes to prevent unstable gradients during long-horizon backpropagation.

After we update the policy π_θ, we use the trajectories collected in the current learning episode to train the value function V_ϕ. The value function network is trained by the following MSE loss:

$$\mathcal{L}_\phi = \mathop{\mathbb{E}}_{\mathbf{s}\in\{\tau_i\}}\left[\|V_\phi(\mathbf{s}) - \tilde{V}(\mathbf{s})\|^2\right], \tag{6}$$

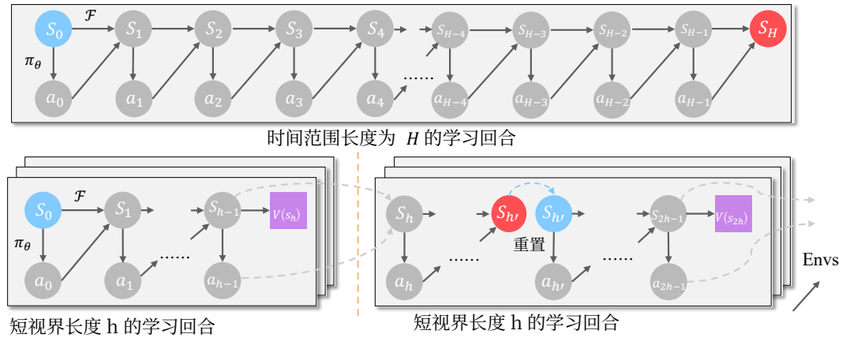

图 3：**BPTT 和 SHAC 的计算图**。**上图**：BPTT 在每个学习回合中通过整个轨迹传播梯度。这会导致噪声损失景观、内存增加以及数值梯度问题。**下图**：SHAC 将轨迹细分为跨学习回合的短优化窗口。这产生了更平滑的替代奖励函数并减少了内存需求，从而能够并行采样许多轨迹。当发生提前终止时，环境会被重置。实线箭头表示保留梯度的计算；虚线箭头表示梯度被切断的位置。

学习回合中的多个较短时域的子窗口（图 3）。子窗口内的多步奖励加上从已学习的 critic 获得的终止值估计，用于改进策略网络。利用可微仿真将梯度通过子窗口内的状态和动作进行反向传播，以提供准确的策略梯度。随后收集轨迹展开，并在每个学习回合中用于学习 critic 网络。

具体而言，我们将每个控制问题建模为一个有限时域马尔可夫决策过程（MDP），包含状态空间 $\mathcal{S}$、动作空间 $\mathcal{A}$、奖励函数 $\mathcal{R}$、转移函数 $\mathcal{F}$、初始状态分布 $\mathcal{D}_{\mathbf{s}_0}$以及任务时间范围 H。在每一步，反馈策略 $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$会计算出一个动作向量$\mathbf{a}_t$。虽然我们的方法并不限制策略必须是确定性或随机的，但在实验中我们采用随机策略以促进额外的探索。具体而言，动作通过 $\mathbf{a}_t \sim \mathcal{N}(\mu_\theta(\mathbf{s}_t), \sigma_\theta(\mathbf{s}_t))$进行采样。转移函数 $\mathcal{F}$由我们的可微仿真建模（见第 3.1 节）。每一步都会收到一个单步奖励 $r_t = \mathcal{R}(\mathbf{s}_t, \mathbf{a}_t)$。该问题的目标是找到能够最大化期望有限时域奖励的策略参数 θ。

我们的方法以 on-policy 模式运行，具体如下：在每个学习回合中，算法从仿真中并行采样N 条短时域 $h \ll H$ 的轨迹 $\{\tau_i\}$，这些轨迹从前一学习回合中轨迹的终止状态继续延伸。随后计算如下策略损失：

$$\mathcal{L}_\theta = -\frac{1}{Nh}\sum_{i=1}^{N}\left[\left(\sum_{t=t_0}^{t_0+h-1}\gamma^{t-t_0}\mathcal{R}(\mathbf{s}_t^i, \mathbf{a}_t^i)\right) + \gamma^h V_\phi(\mathbf{s}_{t_0+h}^i)\right], \tag{5}$$

其中 $\mathbf{s}_t^i$ 和 $\mathbf{a}_t^i$ 分别是第 i条轨迹中步骤 t 的{state}和{action}，而 $\gamma < 1$ 是为稳定训练引入的{discount factor}。当{trajectory sampling}期间发生{task termination}时，会执行重置折扣比率等特殊处理。

为了计算{policy loss} $\frac{\partial \mathcal{L}_\theta}{\partial \theta}$的{gradient}，我们将{simulator}视为{PyTorch computation graph}中的一个{differentiable layer}（其{backward pass}如公式 1 所示），并执行常规的{backpropagation}。我们应用{reparameterization sampling}方法来计算{stochastic policy}的{gradient}。有关{gradient computation}的详细信息，请参阅{Appendix A.1}。随后，我们的算法使用一步 Adam（Kingma & Ba, 2014）来更新{policy}。{differentiable simulator}在此起着关键作用，因为它使我们能够充分利用连接{state}和{action}的底层动力学，并优化{policy}，从而在轨迹内产生更好的短时域{reward}，并为长期性能生成更有希望的终端{state}。需要注意的是，我们在各{learning episodes}之间截断{gradients}，以防止在{long-horizon backpropagation}期间出现不稳定的{gradients}。

在更新策略 π_θ后，我们使用当前学习回合中收集的轨迹来训练价值函数 V_ϕ。价值函数网络通过以下均方误差损失进行训练：

$$\mathcal{L}_\phi = \mathop{\mathbb{E}}_{\mathbf{s}\in\{\tau_i\}}\left[\|V_\phi(\mathbf{s}) - \tilde{V}(\mathbf{s})\|^2\right], \tag{6}$$

Algorithm 1: SHAC (Short-Horizon Actor-Critic) Policy Learning

Initialize policy π_θ, value function V_ϕ, and target value function $V_{\phi'} \leftarrow V_\phi$.
for $learning\ episode \leftarrow 1, 2, ..., M$ **do**
  Sample N short-horizon trajectories of length h by the parallel differentiable simulation from the end states of the previous trajectories.
  Compute the policy loss $\mathcal{L}_\theta$ defined in Eq. 5 from the sampled trajectories and $V_{\phi'}$.
  Compute the analytical gradient $\frac{\partial \mathcal{L}_\theta}{\partial \theta}$ and update the policy π_θ one step with Adam.
  Compute estimated values for all the states in sampled trajectories with Eq. 7.
  Fit the value function V_ϕ using the critic loss defined in Eq. 6.
  Update target value function: $V_{\phi'} \leftarrow \alpha V_{\phi'} + (1-\alpha)V_\phi$.
end for

where $\tilde{V}(\mathbf{s})$ is the estimated value of state $\mathbf{s}$, and is computed from the sampled short-horizon trajectories through a td-λ formulation (Sutton et al., 1998), which computes the estimated value by exponentially averaging different k-step returns to balance the variance and bias of the estimation:

$$\tilde{V}(\mathbf{s}_t) = (1-\lambda)\left(\sum_{k=1}^{h-t-1} \lambda^{k-1} G_t^k\right) + \lambda^{h-t-1} G_t^{h-t}, \tag{7}$$

where $G_t^k = \left(\sum_{l=0}^{k-1} \gamma^l r_{t+l}\right) + \gamma^k V_\phi(\mathbf{s}_{t+k})$ is the k-step return from time t. The estimated value $\tilde{V}(\mathbf{s})$ is treated as constant during critic training, as in regular actor-critic RL methods. In other words, the gradient of Eq. 6 does not flow through the states and actions in Eq. 7.

We further utilize the target value function technique (Mnih et al., 2015) to stabilize the training by smoothly transitioning from the previous value function to the newly fitted one, and use the target value function $V_{\phi'}$ to compute the policy loss (Eq. 5) and to estimate state values (Eq. 7). In addition, we apply observation normalization as is common in RL algorithms, which normalizes the state observation by a running mean and standard deviation calculated from the state observations in previous learning episodes. The pseudo code of our method is provided in Algorithm 1.

Our actor-critic formulation has several advantages that enable it to leverage simulation gradients effectively and efficiently. First, the terminal value function absorbs noisy landscape over long dynamics horizons and discontinuity introduced by early termination into a smooth function, as shown in Figure 2 (Right). This smooth surrogate formulation helps reduce the number of local spikes and alleviates the problem of easily getting stuck in local optima. Second, the short-horizon episodes avoid numerical problems when backpropagating the gradient through deeply nested update chains. Finally, the use of short-horizon episodes allows us to update the actor more frequently, which, when combined with parallel differentiable simulation, results in a significant speed up of training time.

4 Experiments

We design experiments to investigate five questions: (1) How does our method compare to the state-of-the-art RL algorithms on classical RL control tasks, in terms of both sample efficiency and wall-clock time efficiency? (2) How does our method compare to the previous differentiable simulation-based policy learning methods? (3) Does our method scale to high-dimensional problems? (4) Is the terminal critic necessary? (5) How important is the choice of short horizon length h for our method?

4.1 Experiment Setup

To ensure a fair comparison for wall-clock time performance, we run all algorithms on the same GPU model (TITAN X) and CPU model (Intel Xeon(R) E5-2620). Furthermore, we conduct hyperparameter searches for all algorithms and report the performance of the best hyperparameters for each problem. In addition, we report the performance averaged from five individual runs for each algorithm on each problem. The details of the experimental setup are provided in Appendix A.2. We also experiment our method with a fixed hyperparameter setting and with a deterministic policy choice. Due to the space limit, the details of those experiments are provided in Appendix A.4.

4.2 Benchmark Control Problems

For comprehensive evaluations, we select six broad control tasks, including five classical RL tasks across different complexity levels, as well as one high-dimensional control task with a large action

Algorithm 1: SHAC (Short-Horizon Actor-Critic) Policy Learning

Initialize policy π_θ, value function V_ϕ, and target value function $V_{\phi'} \leftarrow V_\phi$.
for $learning\ episode \leftarrow 1, 2, ..., M$ **do**
  Sample N short-horizon trajectories of length h by the parallel differentiable simulation from the end states of the previous trajectories.
  Compute the policy loss $\mathcal{L}_\theta$ defined in Eq. 5 from the sampled trajectories and $V_{\phi'}$.
  Compute the analytical gradient $\frac{\partial \mathcal{L}_\theta}{\partial \theta}$ and update the policy π_θ one step with Adam.
  Compute estimated values for all the states in sampled trajectories with Eq. 7.
  Fit the value function V_ϕ using the critic loss defined in Eq. 6.
  Update target value function: $V_{\phi'} \leftarrow \alpha V_{\phi'} + (1-\alpha)V_\phi$.
end for

˜其中 $V(\mathbf{s})$ 是状态s的估计价值，它是通过对采样的短 horizon 轨迹应用 td-λ 公式（Sutton 等人，1998）计算得出的；该公式通过对不同的 k步回报进行指数平均来计算估计价值，以平衡估计的方差和偏差：

$$\tilde{V}(\mathbf{s}_t) = (1-\lambda)\left(\sum_{k=1}^{h-t-1} \lambda^{k-1} G_t^k\right) + \lambda^{h-t-1} G_t^{h-t}, \tag{7}$$

其中 $G_t^k = \left(\sum_{l=0}^{k-1} \gamma^l r_{t+l}\right) + \gamma^k V_\phi(\mathbf{s}_{t+k})$是从时间 t 开始的 k 步回报。估计值 $\tilde{V}(\mathbf{s})$ 在 Critic 训练期间被视为常数，这与常规的演员 - 评论家 RL 方法一致。换言之，公式 6 的梯度不会流经公式 7 中的状态和动作。

我们进一步利用目标价值函数技术（Mnih 等人，2015），通过将先前的价值函数平滑过渡到新拟合的价值函数来稳定训练，并使用目标价值函数 $V_{\phi'}$ 来计算策略损失（公式 5）并估计状态值（公式 7）。此外，我们采用了强化学习算法中常见的观测归一化方法，该方法通过根据先前学习回合中的状态观测计算出的运行均值和标准差对状态观测进行归一化。我们方法的伪代码见算法 1。

我们的演员 - 评论家公式具有多项优势，使其能够有效且高效地利用仿真梯度。首先，如图 2（右）所示，终端价值函数将长动态时间范围长度内的噪声景观以及由提前终止引入的不连续性吸收为一个平滑函数。这种平滑的代理公式有助于减少局部尖峰的数量，并缓解容易陷入局部最优的问题。其次，短视界回合避免了在通过深层嵌套更新链反向传播梯度时出现的数值问题。最后，使用短视界回合使我们能够更频繁地更新演员，当与并行可微分仿真结合时，可显著加快训练时间。

4 实验

我们设计实验以探究五个问题：(1) 在样本效率和挂钟时间效率方面，我们的方法与经典强化学习控制任务上的最先进强化学习算法相比如何？(2) 我们的方法与之前基于可微仿真的策略学习方法相比如何？(3) 我们的方法能否扩展到高维问题？(4) 终端评价器是否必要？(5) 短视界长度 h 的选择对我们的方法有多重要？

4.1 实验设置

为确保实际运行时间性能的公平比较，我们在相同的 GPU 型号（TITAN X）和 CPU 型号（Intel Xeon(R) E5-2620）上运行所有算法。此外，我们对所有算法进行超参数搜索，并报告每个问题下最佳超参数的性能。另外，我们报告了每种算法在每个问题上五次独立运行的平均性能。实验设置的详细信息见附录 A.2。我们还使用固定超参数设置和确定性策略选择对我们的方法进行了实验。由于篇幅限制，这些实验的详细信息见附录 A.4。

4.2 基准控制问题

为了进行全面评估，我们选择了六个广泛的控制任务，包括五个不同复杂度级别的经典强化学习任务，以及一个具有大动作空间的高维控制任务。

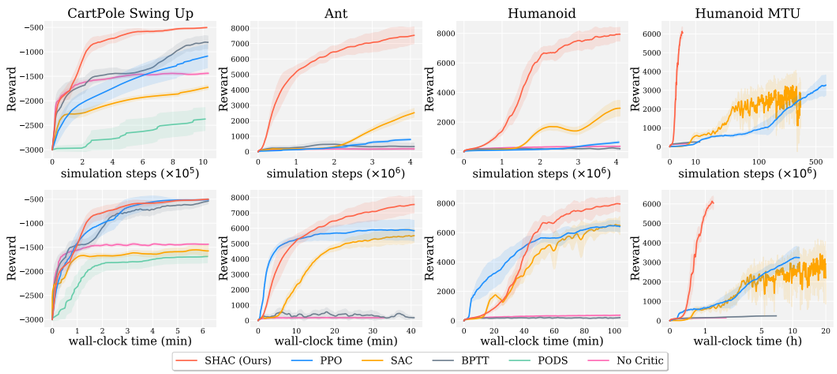

Figure 4: **Learning curves comparison on four benchmark problems.** Each column corresponds to a particular problem, with the top plot evaluating sample efficiency and the bottom plot evaluating wall-clock time efficiency. For better visualization, we truncate all the curves up to the maximal simulation steps/wall-clock time of our method (except for Humanoid MTU), and we provide the full plots in Appendix A.4. Each curve is averaged from five random seeds, and the shaded area shows the standard deviation. SHAC is more sample efficient than all baselines. Model-free baselines are competitive on wall-clock time on pedagogical environments such as the cartpole, but are much less effective as the problem complexity scales.

space. All tasks have stochastic initial states to further improve the robustness of the learned policy. We introduce four representative tasks in the main paper, and leave the others in the Appendix.

Classical Tasks: We select *CartPole Swing Up*, *Ant* and *Humanoid* as three representative RL tasks, as shown in Figure 1. Their difficulty spans from the simplest contact-free dynamics (*CartPole Swing Up*), to complex contact-rich dynamics (*Humanoid*). For *CartPole Swing Up*, we use $H = 240$ as the task horizon, whereas the other tasks use horizons of $H = 1000$.

Humanoid MTU: To assess how our method scales to high-dimensional tasks, we examine the challenging problem of muscle-actuated humanoid control (Figure 1, Right). We use the lower body of the humanoid model from Lee et al. (2019), which contains 152 muscle-tendon units (MTUs). Each MTU contributes one actuated degree of freedom that controls the contractile force applied to the attachment sites on the connected bodies. The task horizon for this problem is $H = 1000$.

To be compatible with differentiable simulation, the reward formulations of each problem are defined as differentiable functions. The details of each task are provided in Appendix A.3.

4.3 Results

Comparison to model-free RL. We compare SHAC with Proximal Policy Optimization (PPO) (Schulman et al., 2017) (on-policy) & Soft Actor-Critic (SAC) (Haarnoja et al., 2018) (off-policy). We use high-performance implementations from *RL games* (Makoviichuk & Makoviychuk, 2021). To achieve state-of-the-art performance, we follow Makoviychuk et al. (2021): all simulation, reward and observation data remain on the GPU and are shared as PyTorch tensors between the RL algorithm and simulator. The PPO and SAC implementations are parallelized and operate on vectorized states and actions. With PPO we used short episode lengths, an adaptive learning rate, and large mini-batches during training to achieve the best possible performance.

As shown in the first row of Figure 4, our method shows significant improvements in sample efficiency over PPO and SAC in three classical RL problems, especially when the dimension of the problem increases (*e.g.*, *Humanoid*). The analytical gradients provided by the differentiable simulation allow us to efficiently acquire the expected policy gradient through a small number of samples. In contrast, PPO and SAC have to collect many Monte-Carlo samples to estimate the policy gradient.

Model-free algorithms typically have a lower per-iteration cost than methods based on differentiable simulation; thus, it makes sense to also evaluate wall-clock time efficiency instead of sample-efficiency alone. As shown in the second row of Figure 4, the wall-clock time performance of PPO, SAC, and our method are much closer than the sample efficiency plot. Interestingly, the training speed of our method is slower than PPO at the start of training. We hypothesize that the target value

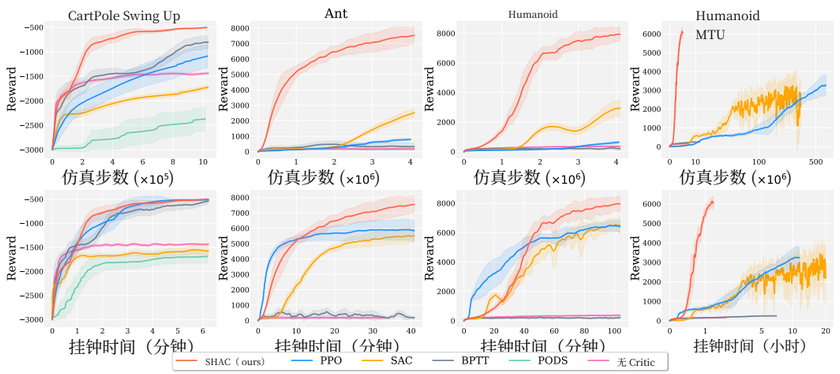

图 4：**四种基准问题上的学习曲线比较**。每一列对应一个特定问题，其中上方图表评估样本效率，下方图表评估挂钟时间效率。为了更好地可视化，我们将所有曲线截断至我们方法的最大仿真步数/挂钟时间（Humanoid MTU 除外），完整图表见附录 A.4。每条曲线均基于五个随机种子取平均，阴影区域表示标准差。SHAC 的样本效率高于所有基线方法。在无接触动力学的教学环境（如 CartPole）中，无模型基线方法在挂钟时间上具有竞争力，但随着问题复杂度增加，其效果显著下降。

所有任务均具有随机初始状态，以进一步提高所学策略的鲁棒性。我们在正文中介绍了四个代表性任务，其余任务见附录。

经典任务：我们选取*CartPole Swing Up*、*Ant* 和*Humanoid* 作为三个代表性强化学习任务，如图 1 所示。它们的难度范围从无接触动力学的最简单情形（*CartPoleSwing Up*）到复杂富接触动力学（*Humanoid*）。对于*CartPole Swing Up*，我们使用 H =240 作为任务时间范围，而其他任务则使用 $H = 1000$的时间范围。

HumanoidMTU：为了评估我们的方法如何扩展到高维任务，我们研究了具有挑战性的肌肉驱动人形控制问题（图 1，右）。我们使用了 Lee et al. (2019) 中人形模型的下半身，该模型包含 152 个肌肉 - 肌腱单元（MTU）。每个 MTU 贡献一个受控自由度，用于控制施加在连接体附着点上的收缩力。该问题的任务时间范围为 $H = 1000$。

为了与可微仿真兼容，每个问题的奖励公式被定义为可微函数。每个任务的详细信息见附录 A.3。

4.3 结果

与无模型强化学习的比较。我们将 SHAC 与近端策略优化（PPO）（Schulman 等，2017）（on-policy）和软演员 - 评论家算法（SAC）（Haarnoja 等，2018）（off-policy）进行比较。我们使用了来自*RL games*（Makoviichuk 和 Makoviychuk，2021）的高性能实现。为了实现最先进性能，我们遵循 Makoviychuk 等（2021）的做法：所有仿真、奖励和观测数据都保留在 GPU 上，并作为 PyTorch 张量在强化学习算法和模拟器之间共享。PPO 和 SAC 的实现已并行化，并作用于向量化状态和动作。在使用 PPO 时，我们采用了较短的回合长度、自适应学习率以及训练期间的大规模小批量，以实现最佳性能。

如图 4 第一行所示，我们的方法在三个经典强化学习问题中，相较于 PPO 和 SAC 展现出显著的样本效率提升，尤其是在问题维度增加时（例如，*Humanoid*）。可微分仿真提供的解析梯度使我们能够通过少量样本高效地获取期望策略梯度。相比之下，PPO 和 SAC 必须收集大量蒙特卡洛样本来估计策略梯度。

无模型算法的每次迭代成本通常低于基于可微仿真的方法；因此，除了样本效率之外，评估挂钟时间效率也是合理的。如图 4 第二行所示，PPO、SAC 与我们方法的实际运行时间性能比样本效率图所示的结果更为接近。有趣的是，在训练初期，我们方法的训练速度慢于 PPO。我们推测，这是因为目标价值

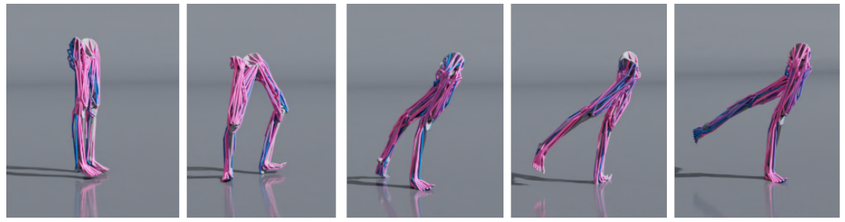

Figure 5: **Humanoid MTU**: A sequence of frames from a learned running gait. The muscle unit color indicates the activation level at the current state.

network in our method is initially requiring sufficient episodes to warm up. We also note that the backward time for our simulation is consistently around $2\times$ that of the forward pass. This indicates that our method still has room to improve its overall wall-clock time efficiency through the development of more optimized differentiable simulators with fast backward gradient computation. We provide a detailed timing breakdown of our method in Appendix A.4.

We observe that our method achieves better policies than RL methods in all problems. We hypothesize that, while RL methods are effective at exploration far from the solution, they struggle to accurately estimate the policy gradient near the optimum, especially in complex problems. We also compare our method to another model-free method REDQ (Chen et al., 2021b) in Appendix A.4.

Comparison with previous gradient-based methods. We compare our approach to three gradient-based learning methods: (1) Backpropagation Through Time (BPTT), which has been widely used in the differentiable simulation literature (Hu et al., 2019; Du et al., 2021), (2) PODS (Mora et al., 2021), and (3) Sample Enhanced Model-Based Policy Optimization (SE-MBPO) (Qiao et al., 2021).

BPTT: The original BPTT method backpropagates gradients over the entire trajectory, which results in exploding gradients as shown in Section 3.2. We modify BPTT to work on a shorter window of the tasks (64 steps for *CartPole* and 128 steps for other tasks), and also leverage parallel differentiable simulation to sample multiple trajectories concurrently to improve its time efficiency. As shown in Figure 4, BPTT successfully optimizes the policy for the contact-free *CartPole Swing Up* task, whereas it falls into local minima quickly in all other tasks involving contact. For example, the policy that BPTT learns for Ant is a stationary position leaning forward, which is a local minimum.

PODS: We compare to the first-order version of PODS, as the second-order version requires the full Jacobian of the state with respect to the whole action trajectory, which is not efficiently available in a reverse-mode differentiable simulator (including ours). Since PODS relies on a trajectory optimization step to optimize an open-loop action sequence, it is not clear how to accommodate early termination where the trajectory length can vary during optimization. Therefore, we test PODS performance only on the *CartPole Swing Up* problem. As shown in Figure 4, PODS quickly converges to a local optimum and is unable to improve further. This is because PODS is designed to be a method with high gradient exploitation but little exploration. Specifically, the line search applied in the trajectory optimization stage helps it converge quickly, but also prevents it from exploring more surrounding space. Furthermore, the extra simulation calls introduced by the line search and the slow imitation learning stage make it less competitive in either sample or wall-clock time efficiency.

SE-MBPO: Qiao et al. (2021) propose to improve a model-based RL method MBPO (Janner et al., 2019) by augmenting the rollout samples using data augmentation that relies on the Jacobian from the differentiable simulator. Although SE-MBPO shows high sample efficiency, the underlying model-based RL algorithm and off-policy training lead to a higher wall-clock time. As a comparison, the officially released code for SE-MBPO takes 8 hours to achieve a reasonable policy in the *Ant* problem used by Qiao et al. (2021), whereas our algorithm takes less than 15 minutes to acquire a policy with the same gait level in our *Ant* problem. Aiming for a more fair comparison, we adapt their implementation to work on our *Ant* problem in our simulator. However, we found that it could not successfully optimize the policy even after considerable hyperparameter tuning. Regardless, the difference in wall-clock time between two algorithms is obvious, and the training time of SE-MBPO is unlikely to be improved significantly by integrating it into our simulation environment. Furthermore, as suggested by Qiao et al. (2021), SE-MBPO does not generalize well to other tasks, whereas our method can be successfully applied to various complexity levels of tasks.

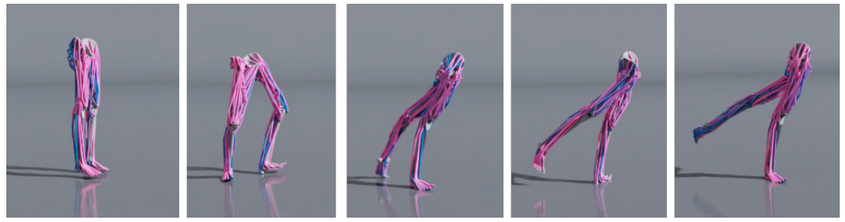

图 5：**Humanoid MTU**：学习到的跑步步态的一系列帧。肌肉单元的颜色表示当前状态下的激活水平。

网络在我们的方法中最初需要足够的回合来预热。我们还注意到，我们仿真的反向时间始终约为前向传递的 $2\times$ 。这表明，通过开发具有快速后向梯度计算的更优化的可微仿真器，我们的方法在整体挂钟时间效率方面仍有提升空间。我们在附录 A.4 中提供了我们方法的详细计时分解。

我们观察到，在所有问题中，我们的方法都比强化学习方法取得了更好的策略。我们假设，虽然强化学习方法在远离解的探索中非常有效，但它们在接近最优解时难以准确估计策略梯度，尤其是在复杂问题中。我们还在附录 A.4 中将我们的方法与另一种无模型方法 REDQ（Chen et al., 2021b）进行了比较。

与先前基于梯度的方法的比较。我们将我们的方法与三种基于梯度的学习方法进行了比较：(1) 时间反向传播（BPTT），该方法已在可微仿真文献中得到广泛应用（Hu et al., 2019；Du et al., 2021）；(2) PODS（Mora et al., 2021）；以及 (3) 样本增强的基于模型的策略优化（SE-MBPO）（Qiaoetal., 2021）。

BPTT：原始的 BPTT 方法在整个轨迹上反向传播梯度，如第 3.2 节所示，这会导致梯度爆炸。我们修改了 BPTT，使其在更短的任务窗口上运行（*CartPole* 为 64 步，其他任务为 128 步），并利用并行可微分仿真同时采样多条轨迹，以提高其时间效率。如图 4 所示，BPTT 成功优化了无接触*CartPole Swing Up* 任务的策略，但在所有涉及接触的其他任务中，它很快陷入了局部极小值。例如，BPTT 为 Ant 学到的策略是一个向前倾斜的静止位置，这是一个局部极小值。

PODS：我们与 PODS 的一阶版本进行了比较，因为其二阶版本需要状态相对于整个动作轨迹的完整雅可比矩阵，而在反向模式可微模拟器（包括我们的模拟器）中无法高效获取该矩阵。由于 PODS 依赖于轨迹优化步骤来优化开环动作序列，因此尚不清楚如何适应提前终止（即在优化过程中轨迹长度可能变化的情况）。因此，我们仅在*CartPole Swing Up* 问题上测试了 PODS 的性能。如图 4 所示，PODS 迅速收敛到局部最优解，且无法进一步改进。这是因为 PODS 被设计为一种具有高梯度利用度但低探索度的方法。具体而言，在轨迹优化阶段应用的线搜索有助于其快速收敛，但也阻碍了其探索更多周围空间。此外，线搜索引入的额外仿真调用以及缓慢的模仿学习阶段，使其在样本效率或挂钟时间效率方面都缺乏竞争力。

SE-MBPO：Qiao 等（2021）提出通过利用可微模拟器的雅可比矩阵进行数据增强，从而扩充 rollout 样本，以改进基于模型的强化学习方法 MBPO（Janner 等，2019）。尽管 SE-MBPO 展现出较高的样本效率，但其底层的基于模型的强化学习算法和离线策略训练导致了更长的挂钟时间。作为对比，SE-MBPO 官方发布的代码在 Qiao 等（2021）所使用的*Ant*问题中需要 8 小时才能获得一个合理的策略，而我们的算法在我们的*Ant* 问题中仅需不到 15 分钟即可获取具有相同步态水平的策略。为了进行更公平的比较，我们调整了他们的实现，使其能在我们模拟器中的*Ant* 问题上运行。然而，我们发现即使经过大量的超参数调优，它仍无法成功优化策略。尽管如此，两种算法在挂钟时间上的差异显而易见，且将 SE-MBPO 集成到我们的仿真环境中也不太可能显著缩短其训练时间。此外，正如 Qiao 等（2021）所指出的，SE-MBPO 难以泛化到其他任务，而我们的方法则可成功应用于各种复杂度水平的任务。

Scalability to high-dimensional problems. We test our algorithm and RL baselines on the *Humanoid MTU* example to compare their scalability to high-dimensional problems. With the large 152-dimensional action space, both PPO and SAC struggle to learn the policy as shown in Figure 4 (Right). Specifically, PPO and SAC learn significantly worse policies after more than 10 hours of training and with hundreds of millions of samples. This is because the amount of data required to accurately estimate the policy gradient significantly increases as the state and action spaces become large. In contrast, our method scales well due to direct access to the more accurate gradients provided by the differentiable simulation with the reparameterization techniques. To achieve the same reward level as PPO, our approach only takes around 35 minutes of training and 1.7M simulation steps. This results in over $17\times$ and $30\times$ wall-clock time improvement over PPO and SAC, respectively, and $382\times$ and $170\times$ more sample efficiency. Furthermore, after training for only 1.5 hours, our method is able to find a policy that has twice the reward of the best-performing policy from the RL methods. A learned running gait is visualized in Figure 5. Such scalability to high-dimensional control problems opens up new possibilities for applying differentiable simulation in computer animation, where complex character models are widely used to provide more natural motion.

Ablation study on the terminal critic. We introduce a terminal critic value in Eq. 5 to account for the long-term performance of the policy after the short episode horizon. In this experiment, we evaluate the importance of this term. By removing the terminal critic from Eq. 5, we get an algorithmic equivalent to BPTT with a short-horizon window and discounted reward calculation. We apply this no-critic variation on all four problems and plot the training curve in Figure 4, denoted by "No Critic". Without a terminal critic function, the algorithm is not able to learn a reasonable policy, as it only optimizes a short-horizon reward of the policy regardless of its long-term behavior.

Study of short horizon length h. The choice of horizon length h is important for the performance of our method. h cannot be too small, as it will result in worse value estimation by td-λ (Eq. 7) and underutilize the power of the differentiable simulator to predict the sensitivity of future performance to the policy weights. On the other hand, a horizon length that is too long will lead to a noisy optimization landscape and less-frequent policy updates. Empirically, we find that a short horizon length $h = 32$ with $N = 64$ parallel trajectories works well for all tasks in our experiments. We conduct a study of short horizon length on the *Ant* task to show the influence of this hyperparameter. We run our algorithm with six short horizon lengths $h = 4, 8, 16, 32, 64, 128$. We set the corresponding number of parallel trajectories $N = 512, 256, 128, 64, 32, 16$ for the variant, such that each one generates the same amount of samples in single learning episode. We run each variant for the same number of episodes $M = 2000$ with 5 individual random seeds. In Figure 6, we report the average reward of the best policies from 5 runs for each variant, as well as the total training time. As expected, the best reward is achieved when $h = 16$ or 32, and the training time scales linearly as h increases.

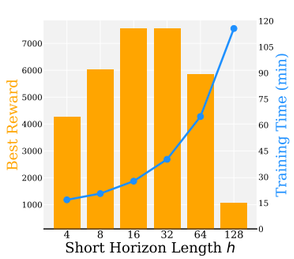

Figure 6: **Study of short horizon length h on Ant problem**. A small h results in worse value estimation. A too large h leads to an ill-posed optimization landscape and longer training time.

5 Conclusion and Future Work

In this work, we propose an approach to effectively leverage differentiable simulation for policy learning. At the core is the use of a critic network that acts as a smooth surrogate to approximate the underlying noisy optimization landscape. In addition, a truncated learning window is adopted to alleviate the problem of exploding/vanishing gradients during deep backward paths. Equipped with the developed parallel differentiable simulation, our method shows significantly higher sample efficiency and wall-clock time efficiency over state-of-the-art RL and gradient-based methods, especially when the problem complexity increases. As shown in our experiments, model-free methods demonstrate efficient learning at the start of training, but SHAC is able to achieve superior performance after a sufficient number of episodes. A compelling future direction for research is how to combine model-free methods with our gradient-based method in order to leverage the strengths of both. Furthermore, in our method, we use a fixed and predetermined short horizon length h throughout the learning process; however, future work may focus on implementing an adaptive short horizon schedule that varies with the status of the optimization landscape.

可扩展至高维问题。我们在*Hu-manoid MTU* 示例上测试了我们的算法和 RL 基线，以比较它们在高维问题上的可扩展性。面对高达 152 维的动作空间，如图 4（右）所示，PPO 和 SAC 都难以学习到有效的策略。具体而言，在经过超过 10 小时的训练并使用数亿样本后，PPO 和 SAC 学到的策略性能显著较差。这是因为随着状态空间和动作空间的增大，准确估计策略梯度所需的数据量会显著增加。相比之下，由于我们的方法能够通过重参数化技术直接访问由可微仿真提供的更精确梯度，因此具有良好的可扩展性。为了达到与 PPO 相同的奖励水平，我们的方法仅需约 35 分钟的训练时间和 1.7M 仿真步数。这使得我们在挂钟时间上分别比 PPO 和 SAC 提升了超过 $17\times$ 和 $30\times$ ，并在样本效率上提高了 $382\times$ 和 $170\times$ 。此外，在仅训练 1.5 小时后，我们的方法就能找到一种策略，其奖励是 RL 方法中表现最佳策略的两倍。图 5 可视化了一种学习到的跑步步态。这种针对高维控制问题的可扩展性，为在计算机动画中应用可微仿真开辟了新的可能性，而计算机动画领域广泛使用复杂的角色模型以提供更自然的运动。

关于终端评价器的消融研究。我们在公式 5 中引入了一个终端评价值，以考虑策略在短视域窗口之后的长期性能。在本实验中，我们评估了该项的重要性。通过从公式 5 中移除终端评价器，我们得到了一个在算法上等价于带有短视域窗口和折扣奖励计算的 BPTT 的算法。我们将这种无 Critic 变体应用于所有四个问题，并在图 4 中绘制了训练曲线，标记为"No Critic"。如果没有终端评论家函数，该算法无法学习到合理的策略，因为它仅优化策略的短视域奖励，而忽略了其长期行为。

短时域长度研究 h。时间范围长度的选择 h 对我们的方法性能至关重要。 h不能太小，否则会导致 td-λ （公式 7）的价值估计变差，并无法充分利用可微模拟器来预测未来性能对策略权重的敏感性。另一方面，过长的时间范围长度会导致优化景观噪声增大以及策略更新频率降低。根据经验，我们发现具有 $N = 64$ 并行轨迹的短视界长度 $h = 32$ 在我们实验的所有任务中均表现良好。我们在 *Ant* 任务上开展了关于短视界长度的研究，以展示该超参数的影响。我们使用六个短视界长度 $h = 4$、8、16、32、64、128 运行我们的算法。对于该变体，我们设置相应的并行轨迹数量 $N = 512$、256、128、64、32、16，使得每个变体在单个学习回合中生成相同数量的样本。我们使用 5 个独立的随机种子，对每个变体运行相同数量的回合 $M = 2000$ 。在图 6 中，我们报告了平均奖励

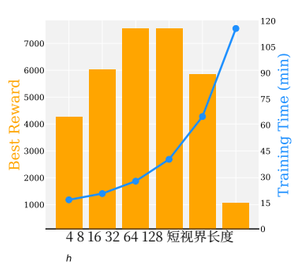

图 6：**短视界长度研究 h 在蚂蚁问题上**。较小的 h 会导致更差的价值估计。过大的 h 会导致病态的优化景观和更长的训练时间。

每个变体在 5 次运行中的最佳策略以及总训练时间。正如预期，当 $h = 16$ 或 32 时获得最佳奖励，且训练时间随 h 的增加而线性增长。

5 结论与未来工作

在这项工作中，我们提出了一种方法，以有效利用可微仿真进行策略学习。其核心是使用一个 critic 网络，作为平滑的代理来近似底层嘈杂的优化景观。此外，我们采用了截断学习窗口，以缓解深层反向路径中的梯度爆炸/消失问题。凭借开发的并行可微分仿真，我们的方法在样本效率和挂钟时间效率上显著优于最先进的强化学习和基于梯度的方法，尤其是在问题复杂度增加时。正如我们的实验所示，无模型方法在训练初期表现出高效的学习能力，但 SHAC 能够在足够数量的回合后实现更优越的性能。一个引人注目的未来研究方向是如何将无模型方法与我们的基于梯度的方法相结合，以利用两者的优势。此外，在我们的方法中，我们在整个学习过程中使用了固定且预定的短视界长度 h ；然而，未来的工作可能会专注于实施一种自适应短视界调度，该调度随优化景观的状态而变化。

ACKNOWLEDGEMENTS

We thank the anonymous reviewers for their helpful comments in revising the paper. The majority of this work was done during the internship of Jie Xu at NVIDIA, and we thank Tae-Yong Kim for his mentorship. This work is also partially supported by Defense Advanced Research Projects Agency (FA8750-20-C-0075).

REFERENCES

Arthur Allshire, Mayank Mittal, Varun Lodaya, Viktor Makoviychuk, Denys Makoviichuk, Felix Widmaier, Manuel Wüthrich, Stefan Bauer, Ankur Handa, and Animesh Garg. Transferring Dexterous Manipulation from GPU Simulation to a Remote Real-World TriFinger. *arXiv preprint arXiv:2108.09779*, 2021.

OpenAI : Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.

Jacob Buckman, Danijar Hafner, George Tucker, Eugene Brevdo, and Honglak Lee. Sample-efficient reinforcement learning with stochastic ensemble value expansion. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, pp. 8234–8244, 2018.

Justin Carpentier and Nicolas Mansard. Analytical derivatives of rigid body dynamics algorithms. In *Robotics: Science and systems (RSS 2018)*, 2018.

Tao Chen, Jie Xu, and Pulkit Agrawal. A system for general in-hand object re-orientation. In *5th Annual Conference on Robot Learning*, 2021a.

Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations*, 2021b.

Ignasi Clavera, Yao Fu, and Pieter Abbeel. Model-augmented actor-critic: Backpropagating through paths. In *International Conference on Learning Representations*, 2020.

Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. 2016.

Filipe de Avila Belbute-Peres, Kevin Smith, Kelsey Allen, Josh Tenenbaum, and J. Zico Kolter. End-to-end differentiable physics for learning and control. In *Advances in Neural Information Processing Systems*, 2018.

Jonas Degrave, Michiel Hermans, Joni Dambre, et al. A differentiable physics engine for deep learning in robotics. *Frontiers in neurorobotics*, pp. 6, 2019.

Marc Peter Deisenroth and Carl Edward Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on International Conference on Machine Learning*, ICML'11, pp. 465–472, 2011. ISBN 9781450306195.

Tao Du, Kui Wu, Pingchuan Ma, Sebastien Wah, Andrew Spielberg, Daniela Rus, and Wojciech Matusik. Diffpd: Differentiable projective dynamics. *ACM Trans. Graph.*, 41(2), nov 2021.

C. Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax - a differentiable physics engine for large scale rigid body simulation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021.

Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pp. 1587–1596. PMLR, 2018.

致谢

我们感谢匿名审稿人对修订论文提出的有益意见。本工作的主要部分是在 Jie Xu 于英伟达实习期间完成的，我们感谢 Tae-Yong Kim 的指导。这项工作还得到了美国国防高级研究计划局（FA8750-20-C-0075）的部分支持。

参考文献

Arthur Allshire, Mayank Mittal, Varun Lodaya, Viktor Makoviychuk, Denys Makoviichuk, Felix Widmaier, Manuel Wüthrich, Stefan Bauer, Ankur Handa, and Animesh Garg. Transferring Dexterous Manipulation from GPU Simulation to a Remote Real-World TriFinger. *arXiv preprint arXiv:2108.09779*, 2021.

OpenAI : Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.

Jacob Buckman, Danijar Hafner, George Tucker, Eugene Brevdo, and Honglak Lee. Sample-efficient reinforcement learning with stochastic ensemble value expansion. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, pp. 8234–8244, 2018.

Justin Carpentier and Nicolas Mansard. Analytical derivatives of rigid body dynamics algorithms. In *Robotics: Science and systems (RSS 2018)*, 2018.

Tao Chen, Jie Xu, and Pulkit Agrawal. A system for general in-hand object re-orientation. In *5th Annual Conference on Robot Learning*, 2021a.

Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations*, 2021b.

Ignasi Clavera, Yao Fu, and Pieter Abbeel. Model-augmented actor-critic: Backpropagating through paths. In *International Conference on Learning Representations*, 2020.

Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. 2016.

Filipe de Avila Belbute-Peres, Kevin Smith, Kelsey Allen, Josh Tenenbaum, and J. Zico Kolter. End-to-end differentiable physics for learning and control. In *Advances in Neural Information Processing Systems*, 2018.

Jonas Degrave, Michiel Hermans, Joni Dambre, et al. A differentiable physics engine for deep learning in robotics. *Frontiers in neurorobotics*, pp. 6, 2019.

Marc Peter Deisenroth and Carl Edward Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on International Conference on Machine Learning*, ICML'11, pp. 465–472, 2011. ISBN 9781450306195.

Tao Du, Kui Wu, Pingchuan Ma, Sebastien Wah, Andrew Spielberg, Daniela Rus, and Wojciech Matusik. Diffpd: Differentiable projective dynamics. *ACM Trans. Graph.*, 41(2), nov 2021.

C. Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax - a differentiable physics engine for large scale rigid body simulation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021.

Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pp. 1587–1596. PMLR, 2018.

Moritz Geilinger, David Hahn, Jonas Zehnder, Moritz Bächer, Bernhard Thomaszewski, and Stelian Coros. Add: Analytically differentiable dynamics for multi-body systems with frictional contact. *ACM Trans. Graph.*, 39(6), November 2020.

Andreas Griewank and Andrea Walther. Introduction to automatic differentiation. *PAMM*, 2(1): 45–49, 2003.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. PMLR, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2019.

Nicolas Heess, Gregory Wayne, David Silver, Timothy Lillicrap, Tom Erez, and Yuval Tassa. Learning continuous control policies by stochastic value gradients. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.

Eric Heiden, David Millard, Erwin Coumans, Yizhou Sheng, and Gaurav S Sukhatme. NeuralSim: Augmenting differentiable simulators with neural networks. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

Yuanming Hu, Jiancheng Liu, Andrew Spielberg, Joshua B Tenenbaum, William T Freeman, Jiajun Wu, Daniela Rus, and Wojciech Matusik. Chainqueen: A real-time differentiable physical simulator for soft robotics. In *2019 International conference on robotics and automation (ICRA)*, pp. 6265–6271. IEEE, 2019.

Yuanming Hu, Luke Anderson, Tzu-Mao Li, Qi Sun, Nathan Carr, Jonathan Ragan-Kelley, and Frédo Durand. Difftaichi: Differentiable programming for physical simulation. *ICLR*, 2020.

Zhiao Huang, Yuanming Hu, Tao Du, Siyuan Zhou, Hao Su, Joshua B. Tenenbaum, and Chuang Gan. Plasticinelab: A soft-body manipulation benchmark with differentiable physics. In *International Conference on Learning Representations*, 2021.

Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), 2019.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in Neural Information Processing Systems*, 32:12519–12530, 2019.

Michael I. Jordan and David E. Rumelhart. Forward models: Supervised learning with a distal teacher. *Cognitive Science*, 16(3):307–354, 1992. ISSN 0364-0213.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, pp. 1008–1014, 2000.

Thanard Kurutach, Ignasi Clavera, Yan Duan, Aviv Tamar, and Pieter Abbeel. Model-ensemble trust-region policy optimization. In *International Conference on Learning Representations*, 2018.

Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47), 2020.

Seunghwan Lee, Moonseok Park, Kyoungmin Lee, and Jehee Lee. Scalable muscle-actuated human simulation and control. *ACM Trans. Graph.*, 38(4), July 2019.

Jacky Liang, Viktor Makoviychuk, Ankur Handa, Nuttapong Chentanez, Miles Macklin, and Dieter Fox. Gpu-accelerated robotic simulation for distributed reinforcement learning. In *Conference on Robot Learning*, pp. 270–282. PMLR, 2018.

Moritz Geilinger, David Hahn, Jonas Zehnder, Moritz Bächer, Bernhard Thomaszewski, and Stelian Coros. Add: Analytically differentiable dynamics for multi-body systems with frictional contact. *ACM Trans. Graph.*, 39(6), November 2020.

Andreas Griewank and Andrea Walther. Introduction to automatic differentiation. *PAMM*, 2(1): 45–49, 2003.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. PMLR, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2019.

Nicolas Heess, Gregory Wayne, David Silver, Timothy Lillicrap, Tom Erez, and Yuval Tassa. Learning continuous control policies by stochastic value gradients. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.

Eric Heiden, David Millard, Erwin Coumans, Yizhou Sheng, and Gaurav S Sukhatme. NeuralSim: Augmenting differentiable simulators with neural networks. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

Yuanming Hu, Jiancheng Liu, Andrew Spielberg, Joshua B Tenenbaum, William T Freeman, Jiajun Wu, Daniela Rus, and Wojciech Matusik. Chainqueen: A real-time differentiable physical simulator for soft robotics. In *2019 International conference on robotics and automation (ICRA)*, pp. 6265–6271. IEEE, 2019.

Yuanming Hu, Luke Anderson, Tzu-Mao Li, Qi Sun, Nathan Carr, Jonathan Ragan-Kelley, and Frédo Durand. Difftaichi: Differentiable programming for physical simulation. *ICLR*, 2020.

Zhiao Huang, Yuanming Hu, Tao Du, Siyuan Zhou, Hao Su, Joshua B. Tenenbaum, and Chuang Gan. Plasticinelab: A soft-body manipulation benchmark with differentiable physics. In *International Conference on Learning Representations*, 2021.

Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), 2019.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in Neural Information Processing Systems*, 32:12519–12530, 2019.

Michael I. Jordan and David E. Rumelhart. Forward models: Supervised learning with a distal teacher. *Cognitive Science*, 16(3):307–354, 1992. ISSN 0364-0213.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, pp. 1008–1014, 2000.

Thanard Kurutach, Ignasi Clavera, Yan Duan, Aviv Tamar, and Pieter Abbeel. Model-ensemble trust-region policy optimization. In *International Conference on Learning Representations*, 2018.

Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47), 2020.

Seunghwan Lee, Moonseok Park, Kyoungmin Lee, and Jehee Lee. Scalable muscle-actuated human simulation and control. *ACM Trans. Graph.*, 38(4), July 2019.

Jacky Liang, Viktor Makoviychuk, Ankur Handa, Nuttapong Chentanez, Miles Macklin, and Dieter Fox. Gpu-accelerated robotic simulation for distributed reinforcement learning. In *Conference on Robot Learning*, pp. 270–282. PMLR, 2018.

Junbang Liang, Ming Lin, and Vladlen Koltun. Differentiable cloth simulation for inverse problems. *Advances in Neural Information Processing Systems*, 32, 2019.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *ICLR (Poster)*, 2016.

Libin Liu and Jessica Hodgins. Learning basketball dribbling skills using trajectory optimization and deep reinforcement learning. *ACM Trans. Graph.*, 37(4), July 2018.

Denys Makoviichuk and Viktor Makoviychuk. RL Games, 2021. URL `https://github.com/Denys88/rl_games/`.

Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu based physics simulation for robot learning. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR.

Miguel Angel Zamora Mora, Momchil Peychev, Sehoon Ha, Martin Vechev, and Stelian Coros. Pods: Policy optimization via differentiable simulation. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 7805–7817. PMLR, 18–24 Jul 2021.

Michael Mozer. A focused backpropagation algorithm for temporal pattern recognition. *Complex Systems*, 3, 01 1995.

D.H. Nguyen and B. Widrow. Neural networks for self-learning control systems. *IEEE Control Systems Magazine*, 10(3):18–23, 1990. doi: 10.1109/37.55119.

OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik's cube with a robot hand, 2019.

Paavo Parmas, Carl Edward Rasmussen, Jan Peters, and Kenji Doya. Pipps: Flexible model-based policy search robust to the curse of chaos. In *International Conference on Machine Learning*, pp. 4065–4074. PMLR, 2018.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Trans. Graph.*, 37(4): 143:1–143:14, July 2018. ISSN 0730-0301.

Xue Bin Peng, Ze Ma, Pieter Abbeel, Sergey Levine, and Angjoo Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Trans. Graph.*, 40(4), July 2021.

Yi-Ling Qiao, Junbang Liang, Vladlen Koltun, and Ming C. Lin. Efficient differentiable simulation of articulated bodies. In *ICML*, 2021.

Junbang Liang, Ming Lin, and Vladlen Koltun. Differentiable cloth simulation for inverse problems. *Advances in Neural Information Processing Systems*, 32, 2019.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *ICLR (Poster)*, 2016.

Libin Liu and Jessica Hodgins. Learning basketball dribbling skills using trajectory optimization and deep reinforcement learning. *ACM Trans. Graph.*, 37(4), July 2018.

Denys Makoviichuk and Viktor Makoviychuk. RL Games, 2021. URL `https://github.com/Denys88/rl_games/`.

Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu based physics simulation for robot learning. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR.

Miguel Angel Zamora Mora, Momchil Peychev, Sehoon Ha, Martin Vechev, and Stelian Coros. Pods: Policy optimization via differentiable simulation. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 7805–7817. PMLR, 18–24 Jul 2021.

Michael Mozer. A focused backpropagation algorithm for temporal pattern recognition. *Complex Systems*, 3, 01 1995.

D.H. Nguyen and B. Widrow. Neural networks for self-learning control systems. *IEEE Control Systems Magazine*, 10(3):18–23, 1990. doi: 10.1109/37.55119.

OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik's cube with a robot hand, 2019.

Paavo Parmas, Carl Edward Rasmussen, Jan Peters, and Kenji Doya. Pipps: Flexible model-based policy search robust to the curse of chaos. In *International Conference on Machine Learning*, pp. 4065–4074. PMLR, 2018.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Trans. Graph.*, 37(4): 143:1–143:14, July 2018. ISSN 0730-0301.

Xue Bin Peng, Ze Ma, Pieter Abbeel, Sergey Levine, and Angjoo Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Trans. Graph.*, 40(4), July 2021.

Yi-Ling Qiao, Junbang Liang, Vladlen Koltun, and Ming C. Lin. Efficient differentiable simulation of articulated bodies. In *ICML*, 2021.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pp. 1889–1897. PMLR, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In Bruce Porter and Raymond Mooney (eds.), *Machine Learning Proceedings 1990*, pp. 216–224. Morgan Kaufmann, San Francisco (CA), 1990. ISBN 978-1-55860-141-3. doi: https://doi.org/10.1016/B978-1-55860-141-3.50030-4.

Richard S Sutton, Andrew G Barto, et al. *Introduction to reinforcement learning*, volume 135. MIT press Cambridge, 1998.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012.

Keenon Werling, Dalton Omens, Jeongseok Lee, Ioannis Exarchos, and C. Karen Liu. Fast and Feature-Complete Differentiable Physics Engine for Articulated Rigid Bodies with Contact Constraints. In *Proceedings of Robotics: Science and Systems*, Virtual, July 2021. doi: 10.15607/RSS.2021.XVII.034.

Jie Xu, Tao Du, Michael Foshey, Beichen Li, Bo Zhu, Adriana Schulz, and Wojciech Matusik. Learning to fly: computational controller design for hybrid uavs with reinforcement learning. *ACM Transactions on Graphics (TOG)*, 38(4):1–12, 2019.

Jie Xu, Tao Chen, Lara Zlokapa, Michael Foshey, Wojciech Matusik, Shinjiro Sueda, and Pulkit Agrawal. An End-to-End Differentiable Framework for Contact-Aware Robot Design. In *Proceedings of Robotics: Science and Systems*, Virtual, July 2021. doi: 10.15607/RSS.2021.XVII.008.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pp. 1889–1897. PMLR, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In Bruce Porter and Raymond Mooney (eds.), *Machine Learning Proceedings 1990*, pp. 216–224. Morgan Kaufmann, San Francisco (CA), 1990. ISBN 978-1-55860-141-3. doi: https://doi.org/10.1016/B978-1-55860-141-3.50030-4.

Richard S Sutton, Andrew G Barto, et al. *Introduction to reinforcement learning*, volume 135. MIT press Cambridge, 1998.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012.

Keenon Werling, Dalton Omens, Jeongseok Lee, Ioannis Exarchos, and C. Karen Liu. Fast and Feature-Complete Differentiable Physics Engine for Articulated Rigid Bodies with Contact Constraints. In *Proceedings of Robotics: Science and Systems*, Virtual, July 2021. doi: 10.15607/RSS.2021.XVII.034.

Jie Xu, Tao Du, Michael Foshey, Beichen Li, Bo Zhu, Adriana Schulz, and Wojciech Matusik. Learning to fly: computational controller design for hybrid uavs with reinforcement learning. *ACM Transactions on Graphics (TOG)*, 38(4):1–12, 2019.

Jie Xu, Tao Chen, Lara Zlokapa, Michael Foshey, Wojciech Matusik, Shinjiro Sueda, and Pulkit Agrawal. An End-to-End Differentiable Framework for Contact-Aware Robot Design. In *Proceedings of Robotics: Science and Systems*, Virtual, July 2021. doi: 10.15607/RSS.2021.XVII.008.

A APPENDIX

A.1 POLICY LOSS GRADIENT COMPUTATION

We minimize the following policy loss to improve the policy network π_θ:

$$\mathcal{L}_\theta = -\frac{1}{Nh}\sum_{i=1}^{N}\left[\Big(\sum_{t=t_0}^{t_0+h-1}\gamma^{t-t_0}\mathcal{R}(\mathbf{s}_t^i,\mathbf{a}_t^i)\Big)+\gamma^h V_\phi(\mathbf{s}_{t_0+h}^i)\right], \tag{8}$$

where N is the number of trajectories, h is the short horizon length, t_0 is the starting time of each trajectory, $\mathbf{s}_t^i$ and $\mathbf{a}_t^i$ are the state and actions at time step t of trajectory τ_i.

To compute its gradient, we treat the differentiable simulator as a differentiable layer (with backward pass shown in Eq. 1) in the computation graph for policy loss $\mathcal{L}_\theta$, and acquiring the gradients $\frac{\partial \mathcal{L}_\theta}{\partial \theta}$ by PyTorch with its reverse-mode computation. Mathematically, we have the following terminal adjoints for each trajectory τ_i

$$\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_{t_0+h}^i} = -\gamma^h \frac{1}{Nh}\frac{\partial V_\phi(\mathbf{s}_{t_0+h}^i)}{\partial \mathbf{s}_{t_0+h}^i} \tag{9}$$

From the last time step t_0+h, we can compute the adjoints in the previous steps $t_0 \le t < t_0+h$ in reverse order:

$$\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_t^i} = -\gamma^{t-t_0}\frac{1}{Nh}\frac{\partial \mathcal{R}(\mathbf{s}_t^i,\mathbf{a}_t^i)}{\partial \mathbf{s}_t^i} + \left(\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_{t+1}^i}\right)\left(\left(\frac{\partial \mathcal{F}}{\partial \mathbf{s}_t^i}\right)+\left(\frac{\partial \mathcal{F}}{\partial \mathbf{a}_t^i}\right)\left(\frac{\partial \pi_\theta(\mathbf{s}_t^i)}{\partial \mathbf{s}_t^i}\right)\right) \tag{10}$$

$$\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{a}_t^i} = -\gamma^{t-t_0}\frac{1}{Nh}\frac{\partial \mathcal{R}(\mathbf{s}_t^i,\mathbf{a}_t^i)}{\partial \mathbf{a}_t^i} + \left(\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_{t+1}^i}\right)\left(\frac{\partial \mathcal{F}}{\partial \mathbf{a}_t^i}\right) \tag{11}$$

From all the computed adjoints, we can compute the policy loss gradient by

$$\frac{\partial \mathcal{L}_\theta}{\partial \theta} = \sum_{i=1}^{N}\sum_{t=t_0}^{t_0+h-1}\left(\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{a}_t^i}\right)\left(\frac{\partial \pi_\theta(\mathbf{s}_t^i)}{\partial \theta}\right) \tag{12}$$

To be noted, for our stochastic policy $\mathbf{a}_t \sim \mathcal{N}(\mu_\theta(\mathbf{s}_t),\sigma_\theta(\mathbf{s}_t))$, we use the reparamterization sampling method, which allows us to compute the gradients $\frac{\partial \pi_\theta(\mathbf{s})}{\partial \theta}$ and $\frac{\partial \pi_\theta(\mathbf{s})}{\partial \mathbf{s}_t}$.

A.2 EXPERIMENT SETUP

To ensure a fair comparison for wall-clock time performance, we run all algorithms on the same GPU model (TITAN X) and CPU model (Intel Xeon(R) E5-2620). Furthermore, we conduct extensive hyperparameter searches for all algorithms and report the performances of the best hyperparameter settings on each problem. For our method, we run for 500 learning episodes for *CartPole Swing Up* problem and for 2000 learning episodes for other five problems. For baseline algorithms, we run each of them for sufficiently long time in order to acquire the policies with the highest rewards. We run each algorithm with five individual random seeds, and report the average performance.

A.2.1 BASELINE ALGORITHM IMPLEMENTATIONS

Policy and Value Network Structures We use multilayer perceptron (MLP) as the policy and value network structures for all algorithms. We use ELU as our activation function for the hidden layers and apply layer normalization to each hidden layer. Since different problems have different state and action dimensionalities, we use different network sizes for different tasks.

Codebase and Implementations For PPO and SAC, we use high-performance implementations of both methods available in RL games (Makoviichuk & Makoviychuk, 2021). To achieve state-of-the-art performance we follow the approach proposed by Makoviychuk et al. (2021) where all the simulation, reward and observation data stays on GPU and is shared as PyTorch tensors between the RL algorithm and the parallel simulator. Both PPO and SAC implementations are parallelized and operate on vectorized states and actions. With PPO we used short episode lengths, an adaptive learning rate, and large mini-batch sizes during training to achieve the maximum possible performance.

For BPTT, the original BPTT requires to backprapagate gradients over the entire trajectory and results in exploding gradients problem. In order to make it work on our long-horizon tasks with

A 附录

A.1 策略损失梯度计算

我们最小化以下策略损失以改进策略网络 π_θ:

$$\mathcal{L}_\theta = -\frac{1}{Nh}\sum_{i=1}^{N}\left[\Big(\sum_{t=t_0}^{t_0+h-1}\gamma^{t-t_0}\mathcal{R}(\mathbf{s}_t^i,\mathbf{a}_t^i)\Big)+\gamma^h V_\phi(\mathbf{s}_{t_0+h}^i)\right], \tag{8}$$

其中 N 是轨迹数量， h 是短视界长度， t_0 是每条轨迹的起始时间，$\mathbf{s}_t^i$ 和$\mathbf{a}_t^i$ 分别是轨迹 τ_i在时间步 t 的状态和动作。

为了计算其梯度，我们将可微模拟器视为策略损失 $\mathcal{L}_\theta$计算图中的一个可微层（其反向传播如公式 1 所示），并通过 PyTorch 的反向模式计算获取梯度 $\frac{\partial \mathcal{L}_\theta}{\partial \theta}$。从数学上讲，对于每条轨迹 τ_i，我们有以下终端伴随变量:

$$\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_{t_0+h}^i} = -\gamma^h \frac{1}{Nh}\frac{\partial V_\phi(\mathbf{s}_{t_0+h}^i)}{\partial \mathbf{s}_{t_0+h}^i} \tag{9}$$

从最后一个时间步 t_0+h 开始，我们可以按逆序计算前几步的伴随变量 $t_0 \le t < t_0+h$:

$$\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_t^i} = -\gamma^{t-t_0}\frac{1}{Nh}\frac{\partial \mathcal{R}(\mathbf{s}_t^i,\mathbf{a}_t^i)}{\partial \mathbf{s}_t^i} + \left(\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_{t+1}^i}\right)\left(\left(\frac{\partial \mathcal{F}}{\partial \mathbf{s}_t^i}\right)+\left(\frac{\partial \mathcal{F}}{\partial \mathbf{a}_t^i}\right)\left(\frac{\partial \pi_\theta(\mathbf{s}_t^i)}{\partial \mathbf{s}_t^i}\right)\right) \tag{10}$$

$$\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{a}_t^i} = -\gamma^{t-t_0}\frac{1}{Nh}\frac{\partial \mathcal{R}(\mathbf{s}_t^i,\mathbf{a}_t^i)}{\partial \mathbf{a}_t^i} + \left(\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{s}_{t+1}^i}\right)\left(\frac{\partial \mathcal{F}}{\partial \mathbf{a}_t^i}\right) \tag{11}$$

利用所有计算出的伴随变量，我们可以通过以下公式计算策略损失梯度

$$\frac{\partial \mathcal{L}_\theta}{\partial \theta} = \sum_{i=1}^{N}\sum_{t=t_0}^{t_0+h-1}\left(\frac{\partial \mathcal{L}_\theta}{\partial \mathbf{a}_t^i}\right)\left(\frac{\partial \pi_\theta(\mathbf{s}_t^i)}{\partial \theta}\right) \tag{12}$$

需要注意的是，对于我们的随机策略 $\mathbf{a}_t \sim \mathcal{N}(\mu_\theta(\mathbf{s}_t),\sigma_\theta(\mathbf{s}_t))$，我们采用了重参数化采样方法，这使得我们能够计算梯度 $\frac{\partial \pi_\theta(\mathbf{s})}{\partial \theta}$ 和 $\frac{\partial \pi_\theta(\mathbf{s})}{\partial \mathbf{s}_t}$。

A.2 实验设置

为确保挂钟时间性能比较的公平性，我们在相同的 GPU 型号（TITAN X）和 CPU 型号（Intel Xeon(R) E5-2620）上运行所有算法。此外，我们对所有算法进行了广泛的超参数搜索，并报告了每个问题上最佳超参数设置的性能。对于我们的方法，我们在*CartPole Swing Up*问题上运行 500 个学习回合，在其他五个问题上运行 2000 个学习回合。对于基线算法，我们将每种算法运行足够长的时间，以获取具有最高奖励的策略。我们使用五个独立的随机种子运行每种算法，并报告平均性能。

A.2.1 基线算法实现

策略与价值网络结构对于所有算法，我们均采用多层感知机（MLP）作为策略与价值网络结构。我们在隐藏层中使用 ELU 作为激活函数，并对每个隐藏层应用层归一化。由于不同问题的状态和动作维度各不相同，因此我们针对不同的任务采用了不同的网络规模。

代码库与实现对于 PPO 和 SAC，我们使用了 RL games（Makoviichuk 和 Makoviychuk，2021）中提供的这两种方法的高性能实现。为了达到最先进的性能，我们遵循 Makoviychuk 等人（2021）提出的方法，即所有的仿真、奖励和观测数据都保留在 GPU 上，并作为 PyTorch 张量在强化学习算法与并行仿真器之间共享。PPO 和 SAC 的实现均经过并行化处理，可对向量化状态和动作进行操作。在使用 PPO 时，我们在训练过程中采用了较短的回合长度、自适应学习率以及较大的小批量大小，以实现尽可能高的性能。

对于 BPTT，原始的 BPTT 需要在整个轨迹上反向传播梯度，从而导致梯度爆炸问题。为了使其能够适用于我们的长时程任务，

complex dynamics, we modify BPTT to work on a shorter window of the tasks (64 steps for *CartPole* and 128 steps for other tasks). We further improve its wall-clock time efficiency by leveraging parallel differentiable simulation to sample multiple trajectories concurrently.

For PODS, due to the lack of released code, we implement our own version of it. Specifically, we implement the first-order version of PODS proposed by Mora et al. (2021) since the second-order version requires the full Jacobian of the state with respect to the whole action trajectory, which is typically not efficiently available in a reverse-mode differentiable simulator (including ours).

For SE-MBPO, we adapt the implementation released by Qiao et al. (2021) into our benchmark problem.

A.2.2 Implementation Details of Our Method

Policy and Value Network Structures We use multilayer perceptron (MLP) as the policy for our method. We use ELU as our activation function for the hidden layers and apply layer normalization to each hidden layer. Since different problems have different state and action dimensionalities, we use different network sizes for different tasks. Basically, we use larger policy and value network structures for problems with higher degrees of freedoms. Empirically we find the performance of our method is insensitive to the size of the network structures. We report the network sizes that are used in our experiments for Figure 4 in Table 1 for better reproducibility. For other problems (*i.e.*, HalfCheetah and Hopper), refer to our released code for the adopted hyperparameters.

Table 1: The network setting of our method on each problem.

Problems	Policy Network Structure	Value Network Structure
CartPole Swing Up	[64, 64]	[64, 64]
Ant	[128, 64, 32]	[64, 64]
Humanoid	[256, 128]	[128, 128]
Humanoid MTU	[512, 256]	[256, 256]

A.2.3 Hyperparameters Settings

Since our benchmark problems span across a wide range of complexity and problem dimentionality, the optimal hyperparameters in expected will be different across problems. We conduct an extensive hyperparameter search for all algorithms, especially for PPO and SAC, and report their best performing hyperparameter settings for fair comparison. For PPO, we adopt the adaptive learning rate strategy implemented in RL games (Makoviichuk & Makoviychuk, 2021). The hyperparameters of PPO and SAC we used in the experiments are reported in Table 2 and 3.

Table 2: The hyperparameter setting of PPO on each problem.

Parameter names	CartPole	Ant	Humanoid	Humanoid MTU
short horizon length h	240	32	32	32
number of parallel environments N	32	2048	1024	1024
discount factor γ	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95
minibatch size	1920	16384	8192	8192
mini epochs	5	5	5	6

For our method, we also conduct hyperparameter searches for each problem, including the short horizon length h, the number of parallel environments N, and the policy and critic learning rates. The learning rates follow a *linear decay* schedule over episodes. We report performance from the

并处理复杂动力学，我们修改了 BPTT，使其仅在任务的较短窗口内运行（*CartPole*为 64 步，其他任务为 128 步）。我们还通过利用并行可微分仿真并发采样多条轨迹，进一步提升了其挂钟时间效率。

对于 PODS，由于缺乏发布的代码，我们实现了自己的版本。具体而言，我们实现了 Mora 等 (2021) 提出的一阶版 PODS，因为二阶版需要状态关于整个动作轨迹的完整雅可比矩阵，而这在反向模式可微模拟器（包括我们的模拟器）中通常无法高效获取。

对于 SE-MBPO，我们将 Qiao 等 (2021) 发布的实现适配到了我们的基准问题中。

A.2.2 我们方法的实现细节

策略与价值网络结构我们使用多层感知机 (MLP) 作为我们方法的策略。我们对隐藏层使用 ELU 作为激活函数，并对每个隐藏层应用层归一化。由于不同问题具有不同的状态和动作维度，我们为不同任务使用不同的网络规模。基本上，对于自由度更高的问题，我们使用更大的策略和价值网络结构。经验表明，我们方法的性能对网络结构的规模不敏感。为了提高可复现性，我们在表 1 中报告了图 4 实验中所使用的网络规模。对于其他问题（即 HalfCheetah 和 Hopper），请参考我们发布的代码以获取所采用的超参数。

表 1：我们的方法在每个问题上的网络设置。

问题	策略网络结构	价值网络结构
CartPole Swing Up	[64, 64]	[64, 64]
Ant	[128, 64, 32]	[64, 64]
Humanoid	[256, 128]	[128, 128]
Humanoid MTU	[512, 256]	[256, 256]

A.2.3 超参数设置

由于我们的基准问题涵盖了广泛的复杂度和问题维度，预期的最优超参数在不同问题之间会有所不同。我们对所有算法进行了广泛的超参数搜索，尤其是针对 PPO 和 SAC，并报告了它们表现最佳的超参数设置以确保公平比较。对于 PPO，我们采用了 RL games（Makoviichuk 和 Makoviychuk，2021）中实现的自适应学习率策略。我们在实验中使用的 PPO 和 SAC 的超参数详见表 2 和表 3。

表 2：PPO 在各问题上的超参数设置。

参数名称	CartPole	Ant	Humanoid	Humanoid MTU
短视界长度 h	240	32	32	32
并行环境数 N	32	2048	1024	1024
折扣因子 γ	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95
小批量大小	1920	16384	8192	8192
小轮次	5	5	5	6

对于我们的方法，我们还针对每个问题进行了超参数搜索，包括短视界长度 h、并行环境数 N，以及策略和评论家学习率。学习率在回合中遵循线性衰减计划。我们报告了来自

Table 3: The hyperparameter setting of SAC on each problem.

Parameter names	**CartPole**	**Ant**	**Humanoid**	**Humanoid MTU**
num steps per episode h	128	128	128	128
number of parallel environments N	32	128	64	256
discount factor γ	0.99	0.99	0.99	0.99
α learning rate	0.005	0.005	0.0002	0.0002
actor learning rate	0.0005	0.0005	0.0003	0.0003
critic learning rate	0.0005	0.0005	0.0003	0.0003
critic τ	0.005	0.005	0.005	0.005
minibatch size	1024	4096	2048	4096
replay buffer size	1e6			
learnable temperature	True			
number of seed steps	2			

best hyperparameter settings in Figure 4, and report the hyperparameters for each problem in Table 4.

Table 4: The optimal hyperparameter setting of our method on each problem.

Parameter names	**CartPole**	**Ant**	**Humanoid**	**Humanoid MTU**
short horizon length h	32			
number of parallel environments N	64			
policy learning rate	0.01	0.002		
critic learning rate	0.001	0.002	0.0005	
target value network α	0.2		0.995	
discount factor γ	0.99			
value estimation λ	0.95			
Adam (β_1, β_2)	$(0.7, 0.95)$			
number of critic training iterations	16			
number of critic training minibatches	4			
number of episodes M	500	2000		

Although the optimal hyperparameters are different for different problems due to the different complexity of the problem and we report the best setting for fair comparison against baseline algorithms, in general, we found the set of hyperparameters in Table 5 works reasonably well on all problems (leading to slightly slower convergence speed in CartPole Swing Up and Ant tasks due to large α for target value network). More analysis is provided in Appendix A.4.3.

A.3 Benchmark Control Problems

We describe more details of our benchmark control problems in this section. We evaluate our algorithm and baseline algorithms on six benchmark tasks, including five classical RL benchmark problems and one high-dimensional problem. We define the reward functions to be differentiable functions in order to incorporate them into our differentiable simulation framework.

A.3.1 CartPole Swing Up

CartPole Swing Up is one of the simplest classical RL control tasks. In this problem, the control policy has to swing up the pole to the upward direction and keeps it in that direction as long as possible. We have 5-dimensional observation space as shown in Table 6 and 1-dimensional action to control the torque applied to the prismatic joint of the cart base.

The single-step reward is defined as:

$$\mathcal{R} = -\theta^2 - 0.1\dot{\theta}^2 - 0.05x^2 - 0.1\dot{x}^2 \tag{13}$$

表 3：SAC 在各问题上的超参数设置。

参数名称	**CartPole**	**Ant**	**Humanoid**	**Humanoid MTU**
每回合步数 h	128	128	128	128
并行环境数 N	32	128	64	256
折扣因子 γ	0.99	0.99	0.99	0.99
α 学习率	0.005	0.005	0.0002	0.0002
演员学习率	0.0005	0.0005	0.0003	0.0003
评论家学习率	0.0005	0.0005	0.0003	0.0003
critic τ	0.005	0.005	0.005	0.005
小批量大小	1024	4096	2048	4096
回放缓冲区大小	1e6			
可学习温度	True			
种子步数	2			

图 4 中最佳超参数设置的性能，并在表 4 中报告了每个问题的超参数。

表 4：我们的方法在每个问题上的最优超参数设置。

参数名称	**CartPole**	**Ant**	**Humanoid**	**Humanoid MTU**
短视界长度 h	32			
并行环境数 N	64			
策略学习率	0.01	0.002		
评论家学习率	0.001	0.002	0.0005	
目标价值网络 α	0.2		0.995	
折扣因子 γ	0.99			
价值估计 λ	0.95			
Adam (β_1, β_2)	$(0.7, 0.95)$			
critic 训练迭代次数	16			
critic 训练小批量数	4			
回合数 M	500	2000		

尽管由于问题复杂度不同，不同问题的最优超参数也有所不同，且我们报告了最佳设置以便与基线算法进行公平比较，但总体而言，我们发现表 5 中的超参数集在所有问题上均表现良好（在 CartPole Swing Up 和 Ant 任务中，由于目标价值网络的 α较大，导致收敛速度略慢）。更多分析见附录 A.4.3。

A.3 基准控制问题

本节将更详细地描述我们的基准控制问题。我们在六个基准任务上评估了我们的算法和基线算法，其中包括五个经典的强化学习基准问题和一个高维问题。我们将奖励函数定义为可微函数，以便将其纳入我们的可微仿真框架中。

A.3.1 CARTPOLE 摆起

CartPole Swing Up 是最简单的经典强化学习控制任务之一。在该问题中，控制策略必须将杆摆至向上方向，并尽可能长时间地保持在该方向。如表 6 所示，我们拥有 5 维观测空间，以及用于控制施加于小车基座移动关节的扭矩的 1 维动作。

单步奖励定义为：

$$\mathcal{R} = -\theta^2 - 0.1\dot{\theta}^2 - 0.05x^2 - 0.1\dot{x}^2 \tag{13}$$

Table 5: A general setting of hyperparameters of our method.

Hyperparameter names	**Values**
short horizon length h	32
number of parallel environments N	64
policy learning rate	0.002
critic learning rate	0.0005
discount factor γ	0.99
value estimation λ	0.95
target value network α	0.995
number of critic training iterations	16
number of critic training minibatches	4

Table 6: Observation vector of CartPole Swing Up problem

Observation	**Degrees of Freedom**
position of cart base: x	1
velocity of the cart base: $\dot{x}$	1
sine and cosine of the pole angle: $\sin(\theta), \cos(\theta)$	2
angular velocity of pole: $\dot{\theta}$	1

The initial state is randomly sampled. The task horizon is 240 steps and there is no early termination in this environment so that it can be used to test the algorithms which are not compatible with early termination strategy.

A.3.2 HALFCHEETAH

In this problem, a two-legged cheetah robot is controlled to run forward as fast as possible. We have 17-dimensional observation space as shown in Table 7 and 6-dimensional action to control the torque applied to each joint.

Table 7: Observation vector of HalfCheetah problem

Observation	**Degrees of Freedom**
height of the base: h	1
rotation angle of the base	1
linear velocity of the base: v	2
angular velocity of the base	1
joint angles	6
joint angle velocities	6

The single-step reward is defined as:

$$\mathcal{R} = v_x - 0.1\|a\|^2, \tag{14}$$

where v_x is the forward velocity.

表 5：我们方法的超参数通用设置。

超参数名称 值	
短视界长度 h	32
并行环境数 N	64
策略学习率	0.002
评论家学习率	0.0005
折扣因子 γ	0.99
价值估计 λ	0.95
目标价值网络 α	0.995
critic 训练迭代次数	16
critic 训练小批量数	4

表 6：CartPole Swing Up 问题的观测向量

观测自由度	
推车底座位置： x	1
推车底座速度： $\dot{x}$	1
杆角度的正弦和余弦：$\sin(\theta)$,$\cos(\theta)$	2
杆的角速度： $\dot{\theta}$	1

初始状态为随机采样。任务时间范围为 240 步，且该环境中不存在提前终止，因此可用于测试不兼容提前终止策略的算法。

A.3.2 HALFCHEETAH

在此问题中，控制双足猎豹机器人尽可能快地向前奔跑。我们拥有如表 7 所示的 17 维观测空间，以及用于控制施加到每个关节扭矩的 6 维动作。

表 7：HalfCheetah 问题的观测向量

观测	**自由度**
底座高度： h	1
底座旋转角度	1
基座线速度： v	2
基座角速度	1
关节角度	6
关节角速度	6

单步奖励定义为：

$$\mathcal{R} = v_x - 0.1\|a\|^2, \tag{14}$$

其中 v_x 是前进速度。

The initial state is randomly sampled. The task horizon is 1000 steps and there is no early termination in this environment.

A.3.3 HOPPER

In this problem, a jumping hopper is controlled to run forward as fast as possible. We have 11-dimensional observation space as shown in Table 8 and 3-dimensional action to control the torque applied to each joint.

Table 8: Observation vector of Hopper problem

Observation	**Degrees of Freedom**
height of the base: h	1
rotation angle of the base θ	1
linear velocity of the base: v	2
angular velocity of the base	1
joint angles	3
joint angle velocities	3

The single-step reward is defined as:

$$\mathcal{R} = \mathcal{R}_v + \mathcal{R}_{height} + \mathcal{R}_{angle} - 0.1\|a\|^2, \tag{15}$$

where $\mathcal{R}_v = v_x$ is the forward velocity, $\mathcal{R}_{height}$ is designed to penalize the low height state, defined by:

$$\mathcal{R}_{height} = \begin{cases} -200\Delta_h^2, & \Delta_h < 0 \\ \Delta_h, & \Delta_h \geq 0 \end{cases} \tag{16}$$

$$\Delta_h = \text{clip}(h + 0.3, -1, 0.3) \tag{17}$$

and the $\mathcal{R}_{angle}$ is designed to encourage the upper body of the hopper to be as upward as possible, defined by

$$\mathcal{R}_{angle} = 1 - \left(\frac{\theta}{30^\circ}\right)^2 \tag{18}$$

The initial state is randomly sampled. The task horizon is 1000 steps and early termination is triggered when the height of the hopper is lower than -0.45m.

A.3.4 ANT

In this problem, a four-legged ant is controlled to run forward as fast as possible. We have 37-dimensional observation space as shown in Table 9 and 8-dimensional action to control the torque applied to each joint.

The single-step reward is defined as:

$$\mathcal{R} = \mathcal{R}_v + 0.1\mathcal{R}_{up} + \mathcal{R}_{heading} + \mathcal{R}_{height}, \tag{19}$$

where $\mathcal{R}_v = v_x$ is the forward velocity, $\mathcal{R}_{up}$ = projection(upward direction) encourages the agent to be vertically stable, $\mathcal{R}_{heading}$ = projection(forward direction) encourages the agent to run straight forward, and $\mathcal{R}_{height} = h - 0.27$ is the height reward.

The initial state is randomly sampled. The task horizon is 1000 steps and early termination is triggered when the height of the ant is lower than 0.27m.

A.3.5 HUMANOID

In this problem, a humanoid robot is controlled to run forward as fast as possible. We have a 76-dimensional observation space as shown in Table 10 and a 21-dimensional action vector to control the torque applied to each joint.

初始状态是随机采样的。任务时间范围为 1000 步，且在此环境中没有提前终止。

A.3.3 HOPPER

在此问题中，控制一个跳跃的 Hopper 尽可能快地向前奔跑。我们拥有如表 8 所示的 11 维观测空间，以及用于控制施加到每个关节的扭矩的 3 维动作。

表 8：Hopper 问题的观测向量

观测	**自由度**
底座高度：h	1
底座旋转角度 θ	1
基座线速度：v	2
基座角速度	1
关节角度	3
关节角速度	3

单步奖励定义为：

$$\mathcal{R} = \mathcal{R}_v + \mathcal{R}_{height} + \mathcal{R}_{angle} - 0.1\|a\|^2, \tag{15}$$

其中 $\mathcal{R}_v = v_x$ 是前进速度，$\mathcal{R}_{height}$ 旨在惩罚低高度状态，其定义为：

$$\mathcal{R}_{height} = \begin{cases} -200\Delta_h^2, & \Delta_h < 0 \\ \Delta_h, & \Delta_h \geq 0 \end{cases} \tag{16}$$

$$\Delta_h = \text{clip}(h + 0.3, -1, 0.3) \tag{17}$$

而 $\mathcal{R}_{angle}$ 旨在鼓励 Hopper 的上半身尽可能向上，其定义为

$$\mathcal{R}_{angle} = 1 - \left(\frac{\theta}{30^\circ}\right)^2 \tag{18}$$

初始状态为随机采样。任务时间范围为 1000 步，当 Hopper 的高度低于 -0.45 米时触发提前终止。

A.3.4 ANT

在此问题中，控制一只四足 Ant 尽可能快地向前奔跑。我们拥有如表 9 所示的 37 维观测空间，以及用于控制施加到每个关节的扭矩的 8 维动作。

单步奖励定义为：

$$\mathcal{R} = \mathcal{R}_v + 0.1\mathcal{R}_{up} + \mathcal{R}_{heading} + \mathcal{R}_{height}, \tag{19}$$

其中 $\mathcal{R}_v = v_x$ 是前进速度，$\mathcal{R}_{up}$ = 投影（向上方向）鼓励代理保持垂直稳定，$\mathcal{R}_{heading}$ = 投影（前进方向）鼓励代理直线向前奔跑，而 $\mathcal{R}_{height} = h - 0.27$ 是高度奖励。

初始状态为随机采样。任务时间范围为 1000 步，当 Ant 的高度低于 0.27m 时触发提前终止。

A.3.5 HUMANOID

在此问题中，控制人形机器人尽可能快地向前奔跑。我们拥有一个如表 10 所示的 76 维观测空间，以及一个用于控制施加到每个关节的扭矩的 21 维动作向量。

Table 9: Observation vector of Ant problem

Observation	**Degrees of Freedom**
height of the base: h	1
rotation quaternion of the base	4
linear velocity of the base: v	3
angular velocity of the base	3
joint angles	8
joint angle velocities	8
up and heading vectors projections	2
actions in last time step	8

Table 10: Observation vector of Humanoid problem

Observation	**Degrees of Freedom**
height of the torso: h	1
rotation quaternion of the torso	4
linear velocity of the torso: v	3
angular velocity of the torso	3
joint angles	21
joint angle velocities	21
up and heading vectors projections	2
actions in last time step	21

The single-step reward is defined as:

$$\mathcal{R} = \mathcal{R}_v + 0.1\mathcal{R}_{up} + \mathcal{R}_{heading} + \mathcal{R}_{height} - 0.002\|\mathbf{a}\|^2, \tag{20}$$

where $\mathcal{R}_v = v_x$ is the forward velocity, $\mathcal{R}_{up}$ is the projection of torso on the upward direction encouraging the agent to be vertically stable, $\mathcal{R}_{heading}$ is the projection of torso on the forward direction encouraging the agent to run straight forward, and $\mathcal{R}_{height}$ is defined by:

$$\mathcal{R}_{height} = \begin{cases} -200\Delta_h^2, & \Delta_h < 0 \\ 10\Delta_h, & \Delta_h \geq 0 \end{cases} \tag{21}$$

$$\Delta_h = \text{clip}(h - 0.84, -1, 0.1) \tag{22}$$

The initial state is randomly sampled. The task horizon is 1000 steps and early termination is triggered when the height of torso is lower than 0.74m.

A.3.6 HUMANOID MTU

This is the most complex problem designed to assess the scalability of the algorithms. In this problem, a lower body of the humanoid model from Lee et al. (2019) is actuated by 152 muscle-tendon units (MTUs). Each MTU contributes one actation degree of freedom that controls the contractile force applied to the attachment sites on the connected bodies. We have a 53-dimensional observation space as shown in Table 11 and a 152-dimensional action vector.

The single-step reward is defined as:

$$\mathcal{R} = \mathcal{R}_v + 0.1\mathcal{R}_{up} + \mathcal{R}_{heading} + \mathcal{R}_{height} - 0.001\|\mathbf{a}\|^2, \tag{23}$$

where $\mathcal{R}_v = v_x$ is the forward velocity, $\mathcal{R}_{up}$ = projection(upward direction) encourages the agent to be vertically stable, $\mathcal{R}_{heading}$ = projection(forward direction) encourages the agent to run straight

表 9：蚂蚁问题的观测向量

观测	**自由度**
底座高度： h	1
基座的旋转四元数	4
基座的线速度： v	3
基座角速度	3
关节角度	8
关节角速度	8
向上和航向向量投影	2
上一时间步动作	8

表 10：人形机器人问题的观测向量

观测	**自由度**
躯干高度： h	1
躯干旋转四元数	4
躯干线速度： v	3
躯干角速度	3
关节角度	21
关节角速度	21
向上和航向向量投影	2
上一时间步动作	21

单步奖励定义为：

$$\mathcal{R} = \mathcal{R}_v + 0.1\mathcal{R}_{up} + \mathcal{R}_{heading} + \mathcal{R}_{height} - 0.002\|\mathbf{a}\|^2, \tag{20}$$

其中 $\mathcal{R}_v = v_x$ 是前进速度， $\mathcal{R}_{up}$ 是躯干在向上方向的投影，用于鼓励智能体保持垂直稳定， $\mathcal{R}_{heading}$ 是躯干在前进方向的投影，用于鼓励智能体直线向前奔跑，而 $\mathcal{R}_{height}$ 定义为：

$$\mathcal{R}_{height} = \begin{cases} -200\Delta_h^2, & \Delta_h < 0 \\ 10\Delta_h, & \Delta_h \geq 0 \end{cases} \tag{21}$$

$$\Delta_h = \text{clip}(h - 0.84, -1, 0.1) \tag{22}$$

初始状态为随机采样。任务时间范围为 1000 个时间步，当躯干高度低于 0.74 米时触发提前终止。

A.3.6 Humanoid MTU

这是为评估算法可扩展性而设计的最复杂问题。在该问题中，Lee et al. (2019) 提出的 Humanoid 模型的下半身由 152 个肌肉 - 肌腱单元（MTU）驱动。每个 MTU 提供一个驱动自由度，用于控制施加到相连物体附着点上的收缩力。如表 11 所示，我们拥有一个 53 维的观测空间和一个 152 维的动作向量。

单步奖励定义为：

$$\mathcal{R} = \mathcal{R}_v + 0.1\mathcal{R}_{up} + \mathcal{R}_{heading} + \mathcal{R}_{height} - 0.001\|\mathbf{a}\|^2, \tag{23}$$

其中 $\mathcal{R}_v = v_x$ 是前进速度， $\mathcal{R}_{up}$ = 投影（向上方向）鼓励智能体保持垂直稳定， $\mathcal{R}_{heading}$ = 投影（前进方向）鼓励智能体直线奔跑

Table 11: Observation vector of Humanoid MTU problem

Observation	**Degrees of Freedom**
height of the pelvis: h	1
rotation quaternion of the pelvis	4
linear velocity of the pelvis: v	3
angular velocity of the pelvis	3
joint angles	22
joint angle velocities	18
up and heading vectors projections	2

forward, and $\mathcal{R}_{height}$ is defined by:

$$\mathcal{R}_{height} = \begin{cases} -200\Delta_h^2, & \Delta_h < 0 \\ 4\Delta_h, & \Delta_h \geq 0 \end{cases} \tag{24}$$

$$\Delta_h = \text{clip}(h - 0.51, -1, 0.05) \tag{25}$$

The initial state is randomly sampled. The task horizon is 1000 steps and early termination is triggered when the height of pelvis is lower than 0.46m.

A.4 More Results

A.4.1 Full Learning Curve Comparison

In Section 4.3, we provide the truncated version of the learning curves in Figure 4 for better visualization. We provide the full version of the learning curves in Figure 7, where the curve for each algorithm continues to the end of the training of that algorithm. In addition, we include the experiment results for the other two classic RL environments, *HalfCheetah* and *Hopper*. From the figure, we can see that our method shows extreme improvements in sample efficiency over other methods and constantly achieves better policies given the same amount of training time.

A.4.2 Wall-Clock Time Breakdown of Training

We provide the detailed wall-clock time performance breakdown of a single learning episode of our method in Table 12. Since we care about the wall-clock time performance of the algorithm, we use this table for a better analysis of which is the time bottleneck in our algorithm. The performance is measured on a desktop with GPU model TITAN X and CPU model Intel Xeon(R) E5-2620 @ 2.10GHz. As shown in the table, the forward simulation and backward simulation time scales up when the dimensionality of the problem increases, while the time spent in critic value function training is almost constant across problems. As expected, the differentiable simulation brings extra overhead for its backward gradient computation. Specifically in our differentiable simulation, the backward computation time is roughly $2\times$ of the forward time. This indicates that our method still has room to improve its overall wall-clock time efficiency through the development of more optimized differentiable simulators with fast backward computation.

A.4.3 Fixed Network Architecture and Hyperparameters

In the previous results, we fine tune the network architectures of policy and value function for each algorithm on each problem to get their maximal performance for a fair comparison. In this section, we test the robustness of our method by using a fixed network architecture and fixed set of hyperparameters (*e.g.* learning rates) to train on all problems. Specifically, we use the same architecture and the hyperparameters used in the *Humanoid* problem, and plot the training curves in Figure 8.

表 11：人形机器人 MTU 问题的观测向量

观测	**自由度**
骨盆高度： h	1
骨盆旋转四元数	4
骨盆的线速度： v	3
骨盆的角速度	3
关节角度	22
关节角速度	18
向上和航向向量投影	2

前进，且 $\mathcal{R}_{height}$ 定义为：

$$\mathcal{R}_{height} = \begin{cases} -200\Delta_h^2, & \Delta_h < 0 \\ 4\Delta_h, & \Delta_h \geq 0 \end{cases} \tag{24}$$

$$\Delta_h = \text{clip}(h - 0.51, -1, 0.05) \tag{25}$$

初始状态为随机采样。任务时间范围为 1000 步，当骨盆高度低于 0.46 米时触发提前终止。

A.4 更多结果

A.4.1 完整学习曲线对比

在 4.3 节中，为了便于可视化，我们提供了图 4 中学习曲线的截断版本。我们在图 7 中提供了学习曲线的完整版本，其中每种算法的曲线均延续至该算法训练结束。此外，我们还包含了另外两个经典强化学习环境的实验结果：*HalfCheetah* 和*Hopper*。从图中可以看出，与其他方法相比，我们的方法在样本效率方面表现出极大的提升，并且在相同的训练时间内始终能获得更优的策略。

A.4.2 训练挂钟时间分解

我们在表 12 中提供了我们方法单个学习回合的详细挂钟时间性能分解。由于我们关注算法的挂钟时间性能，因此利用该表更好地分析我们算法中的时间瓶颈。性能测试在一台配备 GPU 型号 TITAN X 和 CPU 型号 Intel Xeon(R) E5-2620 @ 2.10GHz 的台式机上进行。如表所示，随着问题维度的增加，前向仿真和后向仿真的时间随之增长，而 critic 价值函数训练所花费的时间在不同问题中几乎保持不变。正如预期，可微仿真因其后向梯度计算带来了额外的开销。具体而言，在我们的可微仿真中，后向计算时间约为前向时间的 $2\times$ 。这表明，通过开发具有快速后向计算能力的更优化的可微仿真器，我们的方法在整体挂钟时间效率方面仍有提升空间。

A.4.3 固定网络架构与超参数

在前述结果中，为了进行公平比较，我们针对每种算法在每个问题上微调了策略和价值函数的网络架构，以获得其最佳性能。在本节中，我们通过使用固定的网络架构和固定的超参数集（例如学习率）在所有问题上进行训练，以测试我们方法的鲁棒性。具体而言，我们采用了在*Humanoid* 问题中使用的相同架构和超参数，并在图 8 中绘制了训练曲线。

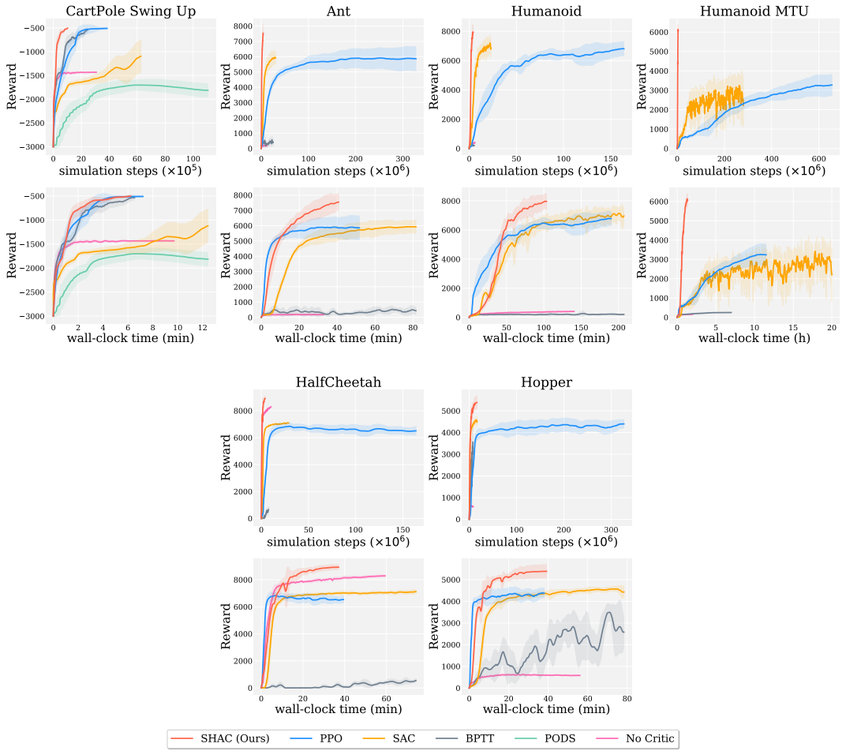

Figure 7: **Full learning curves of algorithms on six benchmark problems.** Each curve is averaged from five random seeds and the shaded area shows the standard deviation. Each problem is shown in a column, with the plot on the top showing the sample efficiency and the curves on the bottom for wall-clock time efficiency. We run our method for 500 learning episodes on CartPole Swing Up, and for 2000 learning episodes for other three tasks. We run other algorithms for sufficiently long time for a comprehensive comparison (either until convergence, or up to 2× of the training time compared to our method on first three problems, and up to 20 hours in Humanoid MTU).

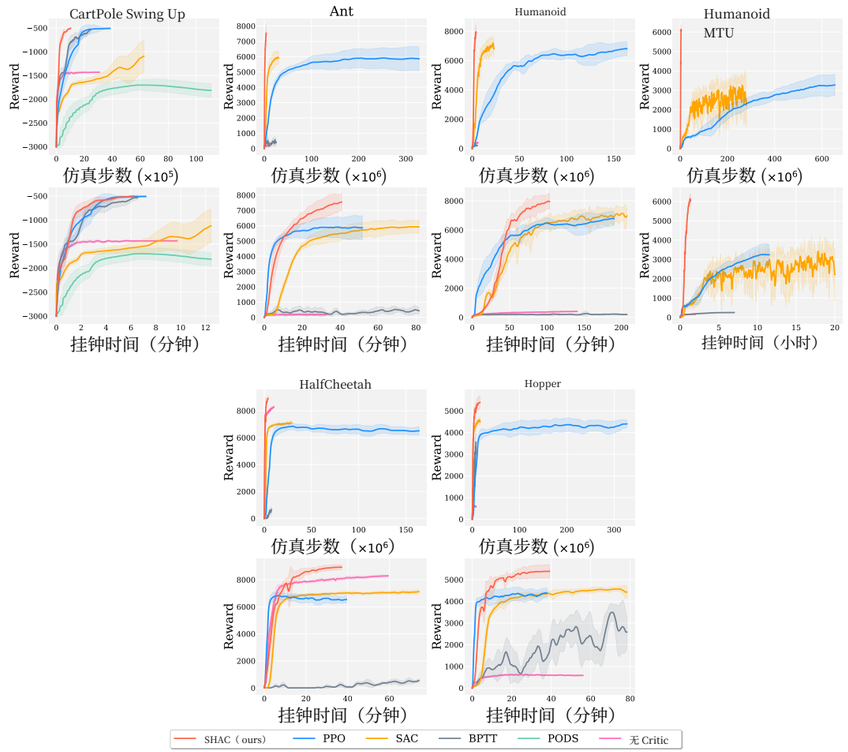

图 7：**六种基准问题上算法的完整学习曲线**。每条曲线由五个随机种子平均得出，阴影区域表示标准差。每个问题占据一列，顶部图表展示样本效率，底部曲线展示挂钟时间效率。我们在 CartPole Swing Up 上运行我们的方法 500 个学习回合，在其他三个任务上运行 2000 个学习回合。我们对其他算法运行足够长的时间以进行全面比较（要么直到收敛，要么在前三个问题上达到与我们方法相比的训练时间的 2× ，或在 Humanoid MTU 中最多运行 20 小时）。

	Forward (s)	**Backward** (s)	**Critic Training** (s)
CartPole SwingUp	0.15	0.23	0.34
Ant	0.25	0.37	0.32
Humanoid	0.91	1.97	0.32
SNU Humanoid	0.82	1.66	0.33

Table 12: **Wall-clock performance breakdown of a single training episode.** The forward stage includes simulation, reward calculation, and observations. Backward includes the simulation gradient calculation and actor update. Critic training, which is specific to our method, is listed individually, and is generally a small proportion of the overall training time.

	前向(s)	**反向**(s)	**Critic 训练**(s)
CartPole SwingUp	0.15	0.23	0.34
Ant	0.25	0.37	0.32
Humanoid	0.91	1.97	0.32
SNU Humanoid	0.82	1.66	0.33

表 12：**单个训练回合的实际耗时性能细分**。前向阶段包括仿真、奖励计算和观测。反向阶段包括仿真梯度计算和 Actor 更新。Critic 训练是我们方法特有的部分，单独列出，通常只占总训练时间的一小部分。

A.4.4 Deterministic Policy

Our method does not constrain the policy to be stochastic or deterministic. We choose to use the stochastic policy in the most experiments for the extra exploration that it provides. In this section, we experiment our method with the deterministic policy choice. Specifically, we change the policy of our method in each problem from stochastic policy to deterministic policy while keeping all

A.4.4 确定性策略

我们的方法并不限制策略必须是随机策略或确定性策略。在大多数实验中，我们选择使用随机策略，因为它能提供额外的探索能力。在本节中，我们在确定性策略的选择下对我们的方法进行了实验。具体而言，我们将每个问题中我们方法的策略从随机策略改为确定性策略，同时保持所有

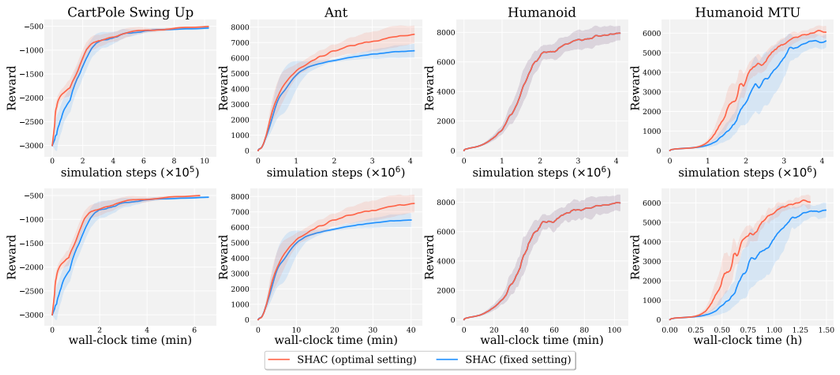

Figure 8: **Learning curves of our method with fixed network architectures and learning rates.** We use the same network architectures and learning rates used in *Humanoid* problem on all other problems, and plot the training curves comparison with the ones using optimal settings. The plot shows that our method still performs reasonably well with the fixed network and learning rates settings.

other hyperparameters such as network dimensions and learning rates the same as shown in Table 4. The training curves of the deterministic policy is plotted in Figure 9. The results show that our method works reasonably well with deterministic policy, and sometimes the deterministic policy even outperforms the stochastic policy (*e.g. Humanoid*). The small performance drop on the *Ant* problem comes from one single random seed (out of five) which results in a sub-optimal policy.

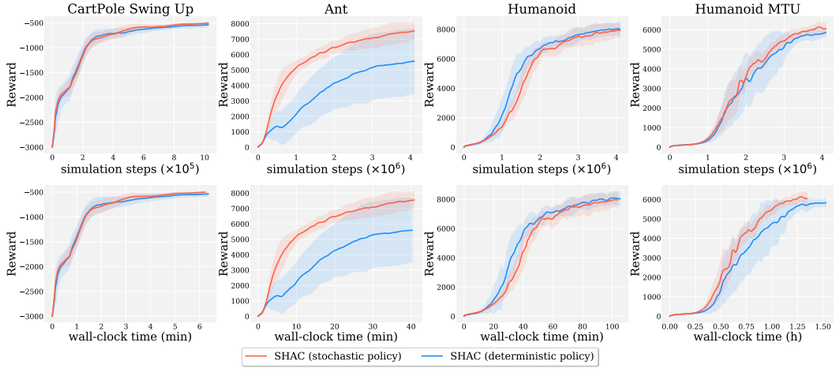

Figure 9: **Learning curves of our method with deterministic policy.** We test our method with deterministic policy choice. We use the same network sizes and the hyperparameters as used in the stochastic policy and remove the policy output stochasticity. We run our method on each problem with five individual random seeds. The results show that our method with deterministic policy works reasonably well on all problems, and sometimes the deterministic policy even outperforms the stochastic policy (*e.g., Humanoid*). The small performance drop on the *Ant* problem comes from one single seed (out of five) which results in a sub-optimal policy.

A.4.5 Comparison to REDQ

In this section, we compare our method to another advanced model-free reinforcement learning algorithm, namely REDQ (Chen et al., 2021b). REDQ is an off-policy method and is featured by its high sample efficiency. We experiment the officially released code of REDQ on four of our benchmark problems: *CartPole Swing Up, Ant, Humanoid,* and *Humanoid MTU*, and provide the comparison between our method and REDQ in Figure 10. From the plots, we see that REDQ

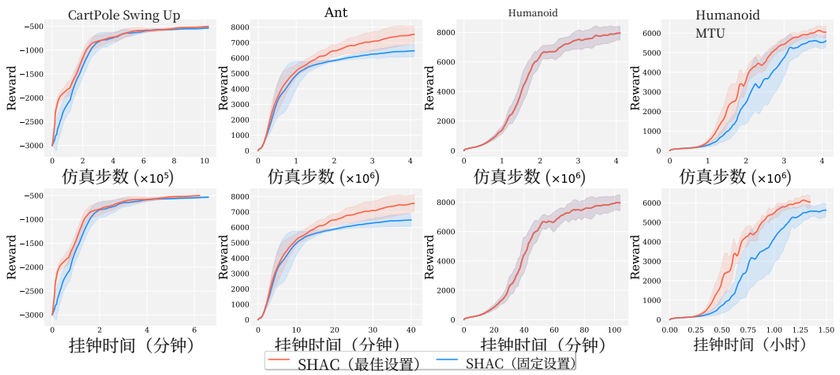

图 8：**在固定网络架构和学习率下，我们方法的学习曲线**。在所有其他问题上，我们使用了与 *Humanoid* 问题相同的网络架构和学习率，并绘制了训练曲线以与使用最优设置的结果进行对比。图表显示，即使在固定的网络和学习率设置下，我们的方法仍然表现良好。

其他超参数（如网络维度和学习率）保持与表 4 所示一致。确定性策略的训练曲线绘制于图 9。结果表明，我们的方法在确定性策略下表现良好，有时甚至优于随机策略（例如 *Humanoid*）。在*Ant*问题上出现的微小性能下降源于五个随机种子中的一个，该种子导致了一个次优策略。

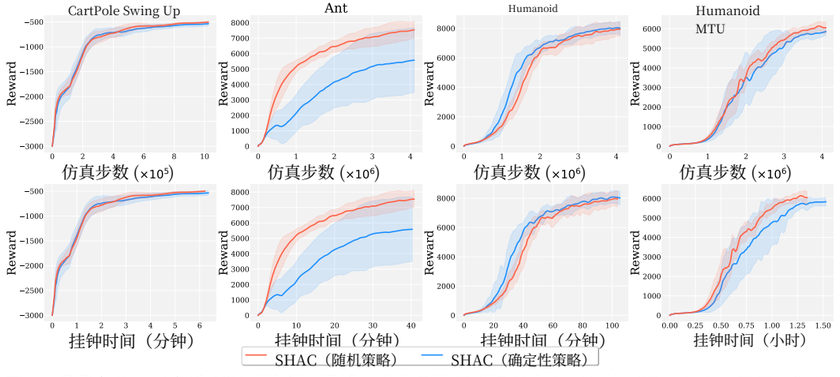

图 9：**我们方法在确定性策略下的学习曲线**。我们测试了选择确定性策略的情况。我们使用了与随机策略相同的网络规模和超参数，并移除了策略输出的随机性。我们在每个问题上使用五个独立的随机种子运行我们的方法。结果表明，采用确定性策略的我们的方法在所有问题上均表现良好，有时甚至优于随机策略（例如*Humanoid*）。在*Ant* 问题上出现的微小性能下降源于五个种子中的一个，该种子导致了一个次优策略。

A.4.5 与 REDQ 的比较

在本节中，我们将我们的方法与另一种先进的无模型强化学习算法 REDQ（Chen et al., 2021b）进行比较。REDQ 是一种异策略方法，以其高样本效率为特点。我们在四个基准问题上实验了 REDQ 官方发布的代码：*CartPole Swing Up*、*Ant*、*Humanoid*和*Humanoid MTU*，并在图 10 中提供了我们的方法与 REDQ 的对比。从图中可以看出，REDQ

demonstrates its high sample efficiency in the *CartPole Swing Up* and *Ant* problems due to its off-policy updates and the randomized Q function ensembling strategy. However, REDQ is extremely slow in wall-clock training time. For example, in *Ant* problem, REDQ takes 27 hours to achieve 5000 rewards while SHAC only takes 13 minutes to achieve the same reward. Such huge difference in wall-clock time performance renders REDQ a less attractive method than SHAC in the settings where the policy is learned in a fast simulation. Furthermore, REDQ fails to train on our high-dimensional problems, *Humanoid* and *Humanoid MTU* problems.

A.4.6 Comparison to Model-Based RL

In this section, we compare our method to a model-based reinforcement learning algorithm. In a typical model-based reinforcement learning algorithm, the learned model can be used in two ways: (1) data augmentation, (2) policy gradient estimation with backpropagation through a differentiable model.

In the data augmentation category, the learned model is used to provide rollouts around the observed environment transitions, as introduced in the DYNA algorithm (Sutton, 1990). The core idea being that the model can be used to provide additional training samples in the neighborhood of observed environment data. The core idea in DYNA was extended by STEVE (Buckman et al., 2018) and MBPO (Janner et al., 2019), which use interpolation between different horizon predictions compared to a single step roll-out in DYNA. However, the performance of such approaches rapidly degrades with increasing model error. Notably, as noted by Janner et al. (2019), empirically the one-step rollout is a very strong baseline to beat in part because error in model can undermine the advantage from model-based data-augmentation.

In contrast the model can be used for policy-value gradient estimation, which originates from Nguyen & Widrow (1990) and Jordan & Rumelhart (1992). This idea has been extended multi-ple times such as in PILCO (Deisenroth & Rasmussen, 2011), SVG (Heess et al., 2015), and MAAC (Clavera et al., 2020). Despite the core method being very similar to the original idea, the difference between recent methods such as SVG and MAAC emerges in the details of what data is used in estimation. SVG uses only real samples, while MAAC uses both real and model roll-outs. But most of these methods still rely on model error being low for policy and value computation.

While SHAC resembles these gradient-based methods, our model usage fundamentally differs from previous approaches. SHAC is the first method that combines actor-critic methods with analytical gradient computation from a differentiable simulator model. The analytical gradient solves the model learning issues which plague methods both in model-based data-augmentation and direct gradient estimators. At the same time, SHAC prevents exploding/vanishing gradients in BPTT through truncated roll-outs and a terminal critic function. SHAC offers improvement over prior model-based methods in both improved efficiency as well as scale-up to higher dimensional problems, beyond the ones reported in these methods.

To further illustrate the advantage of our method over model-based RL, we conduct the experiments to compare to Model-Based Policy Optimization (MBPO) (Janner et al., 2019). We run the officially released code of MBPO on four of our benchmark problems: *CartPole Swing Up, Ant, Humanoid,* and *Humanoid MTU*, and provide the comparison between our method and MBPO in Figure 10. As shown in the figure, the MBPO with the default parameters does not work well on our benchmark problems except *CartPole Swing Up*. We hypothesize that it is because of the hyperparameter sensitivity of model-based RL method. Although a fair comparision can be further conducted by tuning the hyperparameters of MBPO, an implicit comparison can be made through the comparison with REDQ. In REDQ paper (Chen et al., 2021b), it is reported that REDQ is up to 75% faster in wall-clock time than MBPO. This indirectly indicates that our method has significant advantages in the wall-clock time efficiency over MBPO.

A.5 More Optimization Landscape Analysis

In Figure 2, we show the original landscape of the problem and the surrogate landscape defined by SHAC. However the smoothness of the surrogate landscape sometimes does not necessarily results in a smooth gradient when the evaluation trajectory is stochastic and the gradient is computed by averaging over policy stochasticity (Parmas et al., 2018). To further analyze the gradient of the landscape, we plot the comparison of the computed single-weight analytical gradient and the single-

由于其异策略更新和随机化 Q 函数集成策略，在*CartPole Swing Up* 和*Ant* 问题中展现了其高样本效率。然而，REDQ 在挂钟训练时间上极其缓慢。例如，在*Ant* 问题中，REDQ 需要 27 小时才能达到 5000 奖励，而 SHAC 仅需 13 分钟即可达到相同奖励。这种巨大的实际运行时间性能差异使得 REDQ 在策略通过快速仿真学习的场景中，成为比 SHAC 吸引力较低的方法。此外，REDQ 无法在我们的低维问题上进行训练，即*Humanoid* 和*Humanoid MTU* 问题。

A.4.6 与基于模型的强化学习的比较

在本节中，我们将我们的方法与一种基于模型的强化学习算法进行比较。在典型的基于模型的强化学习算法中，所学模型可用于两种方式：(1) 数据增强，(2) 通过对可微模型进行反向传播来进行策略梯度估计。

在数据增强类别中，所学模型用于在观测到的环境转移周围提供 rollout，正如 DYNA 算法（Sutton, 1990）中所引入的那样。其核心思想是，模型可用于在观测到的环境数据邻域内提供额外的训练样本。DYNA 中的这一核心思想被 STEVE（Buckman 等，2018）和 MBPO（Janner 等，2019）所扩展，它们利用不同 horizon predictions 之间的插值，而非 DYNA 中的单步 rollout。然而，此类方法的性能会随着模型误差的增加而迅速下降。值得注意的是，正如 Janner 等（2019）所指出的，从经验上看，单步 rollout 是一个非常强大的基线，难以超越，部分原因在于模型误差可能会削弱基于模型的数据增强所带来的优势。

相比之下，该模型可用于策略 - 价值梯度估计，这一概念源于 Nguyen & Widrow (1990) 和 Jordan & Rumelhart (1992)。这一思想已被多次扩展，例如在 PILCO (Deisenroth& Rasmussen, 2011)、SVG (Heesset al., 2015) 和 MAAC (Clavera et al., 2020) 中。尽管核心方法与原始思想非常相似，但近期方法（如 SVG 和 MAAC）之间的差异体现在估计中所用数据的细节上。SVG 仅使用真实样本，而 MAAC 则同时使用真实样本和模型 rollout。然而，这些方法中的大多数仍然依赖于较低的模型误差来进行策略和价值计算。

虽然 SHAC 与这些基于梯度的方法相似，但我们的模型用法与以往方法有着根本性的不同。SHAC 是首个将演员 - 评论家方法与来自可微仿真器模型的解析梯度计算相结合的方法。解析梯度解决了困扰基于模型的数据增强方法和直接梯度估计器的模型学习问题。同时，SHAC 通过截断 rollout 和终端评论家函数，防止了 BPTT 中的梯度爆炸/消失。相较于先前的基于模型的方法，SHAC 在提高效率以及扩展到更高维问题（超出这些方法所报道的范围）方面均提供了改进。

为了进一步说明我们的方法相较于基于模型的强化学习（RL）的优势，我们进行了实验，将其与基于模型的策略优化（MBPO）（Janner 等，2019）进行比较。我们在四个基准问题上运行了 MBPO 官方发布的代码：*CartPole Swing Up*、*Ant*、*Humanoid*，以及*Humanoid MTU*，并在图 10 中提供了我们的方法与 MBPO 的对比结果。如图所示，使用默认参数的 MBPO 在我们的基准问题上表现不佳，除了*CartPole Swing Up*。我们推测这是由于基于模型的 RL 方法存在超参数敏感性。虽然可以通过调整 MBPO 的超参数来进行更公平的比较，但通过与 REDQ 的对比也可以进行一种隐含的比较。在 REDQ 论文（Chen et al., 2021b）中报道，REDQ 的挂钟时间比 MBPO 快高达 75%。这间接表明，我们的方法在挂钟时间效率方面相对于 MBPO 具有显著优势。

A.5 更多优化景观分析

在图 2 中，我们展示了问题的原始景观以及由 SHAC 定义的替代景观。然而，当评估轨迹具有随机性且梯度是通过对策略随机性进行平均计算得出时（Parmas 等，2018），替代景观的平滑性并不一定意味着梯度也是平滑的。为了进一步分析景观的梯度，我们绘制了计算得到的单权重解析梯度与单

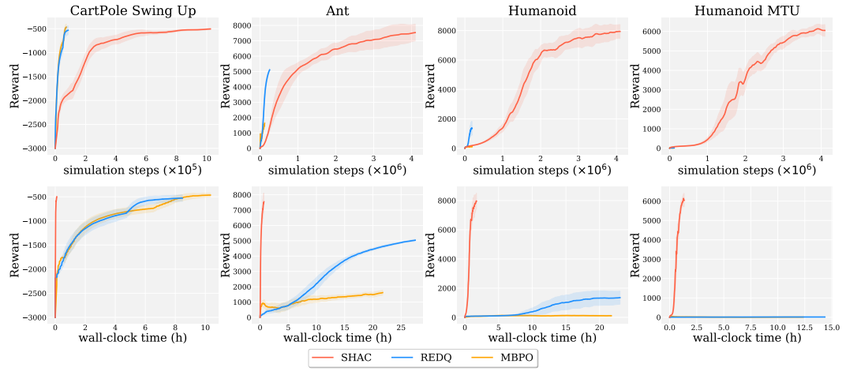

Figure 10: **Comparison with REDQ and MBPO.** We compare SHAC to REDQ and MBPO. While REDQ has very excellent sample efficiency, its wall-clock time efficiency is very poor (102× slower than SHAC in *CartPole Swing Up* and 162× slower than SHAC in *Ant*), which renders it as a less attractive method than SHAC in the settings where the policy is learned in a fast simulation. The offical MBPO code works well on the *CartPole Swing Up* problem while works even worse than REDQ on other three tasks. Furthermore, both REDQ and MBPO fail to train on our high-dimensional problems, *Humanoid* and *Humanoid MTU*, with default hyperparameters.

weight gradient computed by finite difference of the landscape (shown in Figure 2 Right) in Figure 11. The plot demonstrates that our computed gradient is well-behaved and close enough to the finite difference gradient.

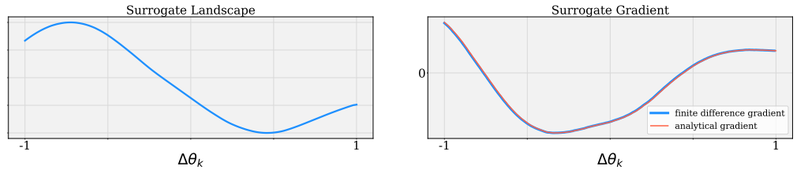

Figure 11: **Single-weight landscape and gradient of the SHAC loss.** In the left figure, we select one single weight from a policy and change its value by $\Delta\theta_k \in [-1, 1]$ to plot the task loss landscapes of SHAC w.r.t. one policy parameter (same as Figure 2 right). The short horizon length is $h = 32$. Each policy variant is evaluated by 128 stochastic trajectories and we set the same random seed before the evaluation of each policy variant. In the right figure, we plot the finite difference gradient and the analytical gradient along that single weight axis. It shows that the analytical gradient is well-behaved and close enough to the finite difference gradient.

Furthermore, we compare the gradient of the BPTT method and the gradient of SHAC in Figure 12, where we picked six different policies during the training for the *Humanoid* problem, and plot the gradient value distribution of BPTT method and SHAC method for each of them. From Figure 12, we can clearly see that the gradient explosion problem exists in the BPTT method while the gradient acquired by SHAC is much more stable throughout the training process.

To better visualize the landscape, we also compare the landscape surfaces in Figure 13. Specifically, we experiment on the landscapes for the *Humanoid* and *Ant* problems. For *Humanoid*, we select a policy during training, and for *Ant* we use the optimal policy from training for study. We randomly select two directions $\vec{\theta}_1$ and $\vec{\theta}_2$ in the policy network parameter space, and evaluate the policy variations along these two directions with weights $\theta' = \theta + k_1\vec{\theta}_1 + k_2\vec{\theta}_2, k_1, k_2 \in [-1, 1]$. The figure shows that the short-horizon episodes and the terminal value critic of SHAC produce a surrogate landscape which is an approximation of the original landscape but is much smoother.

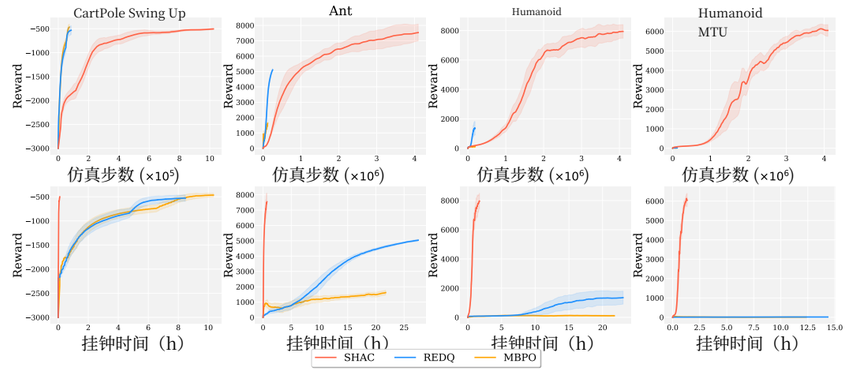

图 10：**与 REDQ 和 MBPO 的比较**。我们将 SHAC 与 REDQ 和 MBPO 进行了比较。虽然 REDQ 具有极佳的样本效率，但其挂钟时间效率非常差（在*CartPole Swing Up* 中比 SHAC 慢102× 倍，在*Ant*中比 SHAC 慢 162× 倍），这使得在策略通过快速仿真进行学习的场景中，REDQ 不如 SHAC 具有吸引力。官方的 MBPO 代码在*CartPole SwingUp* 问题上表现良好，但在其他三个任务上的表现甚至不如 REDQ。此外，使用默认超参数时，REDQ 和 MBPO 均无法在我们的<high-dimensional problems>高维问题</high-dimensional problems>（即 *Humanoid* 和*Humanoid MTU*）上完成训练。

权重梯度（通过景观的有限差分计算得出，见图 2 右侧）的对比图（图 11）。该图表明，我们计算出的梯度表现良好，且与有限差分梯度非常接近。

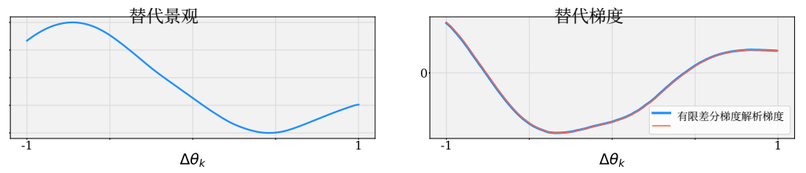

图 11：**SHAC 损失的单权重景观与梯度**。在左图中，我们从策略中选取单个权重，并将其值改变 $\Delta\theta_k \in [-1, 1]$ 以绘制 SHAC 关于单一策略参数的任务损失景观（与图 2 右侧相同）。短视界长度为 $h = 32$。每个策略变体通过 128 条随机轨迹进行评估，且在评估每个策略变体前设置相同的随机种子。在右图中，我们绘制了沿该单权重轴的有限差分梯度和解析梯度。结果表明，解析梯度表现良好，且与有限差分梯度足够接近。

此外，我们在图 12 中比较了 BPTT 方法的梯度与 SHAC 的梯度，其中我们针对 *Humanoid* 问题的训练过程选取了六个不同的策略，并绘制了每种策略下 BPTT 方法与 SHAC 方法的梯度值分布。从图 12 可以清楚地看到，BPTT 方法中存在梯度爆炸问题，而 SHAC 获得的梯度在整个训练过程中要稳定得多。

为了更好地可视化景观，我们还在图 13 中比较了景观曲面。具体而言，我们针对*Humanoid* 和 *Ant* 问题进行了景观实验。对于*Humanoid*，我们选取了训练过程中的一个策略；而对于*Ant* ，我们则使用训练中得到的最优策略进行研究。我们在策略网络参数空间中随机 $\vec{\theta}_1$ $\vec{\theta}_2$选择两个方向 θ_1 和 θ_2 ，并沿这两个方向评估权重为 $\theta' = \theta + k_1\vec{\theta}_1 + k_2\vec{\theta}_2$ k_1 $k_2 \in [-1, 1]$,的策略变化。该图表明，SHAC 的短视界回合和终端价值评判网络生成了一个替代景观，它是对原始景观的近似，但更加平滑。

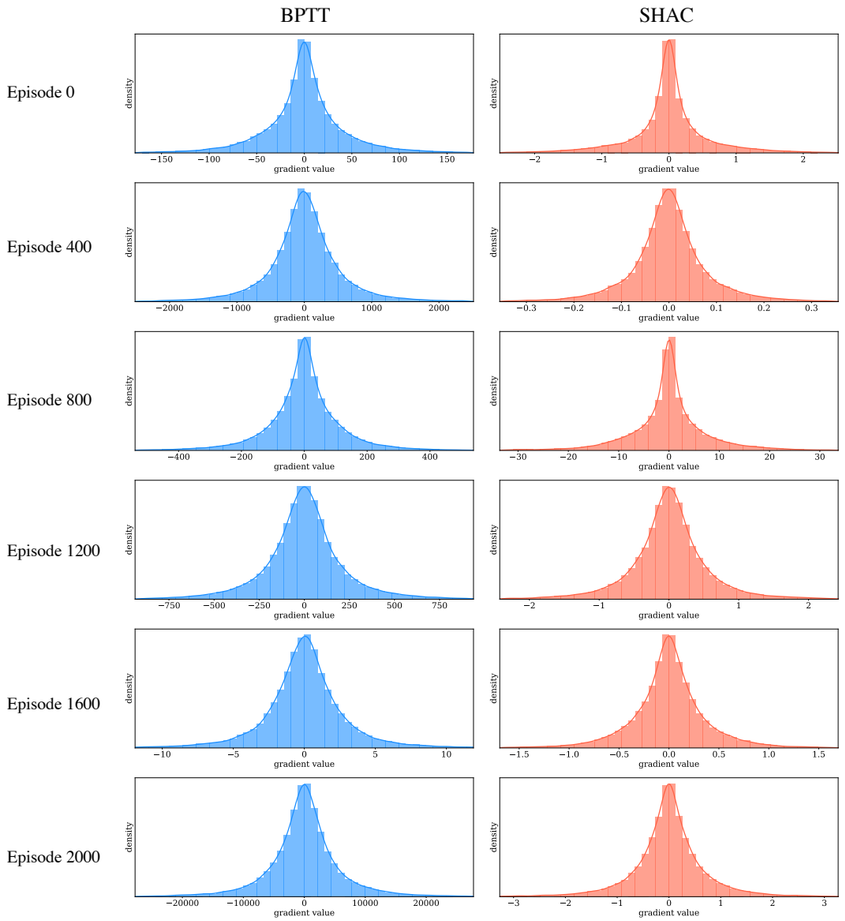

Figure 12: **Gradient distribution evolving over training process.** We select six policies from different episodes during the training process for the *Humanoid* problem and plot the gradient value distribution of BPTT (left) and SHAC (right). Note the x-axis scales on the plots. The plots show that the gradient computed from BPTT suffers from gradient explosion, whereas the analytical gradient computed from our method is much more stable.

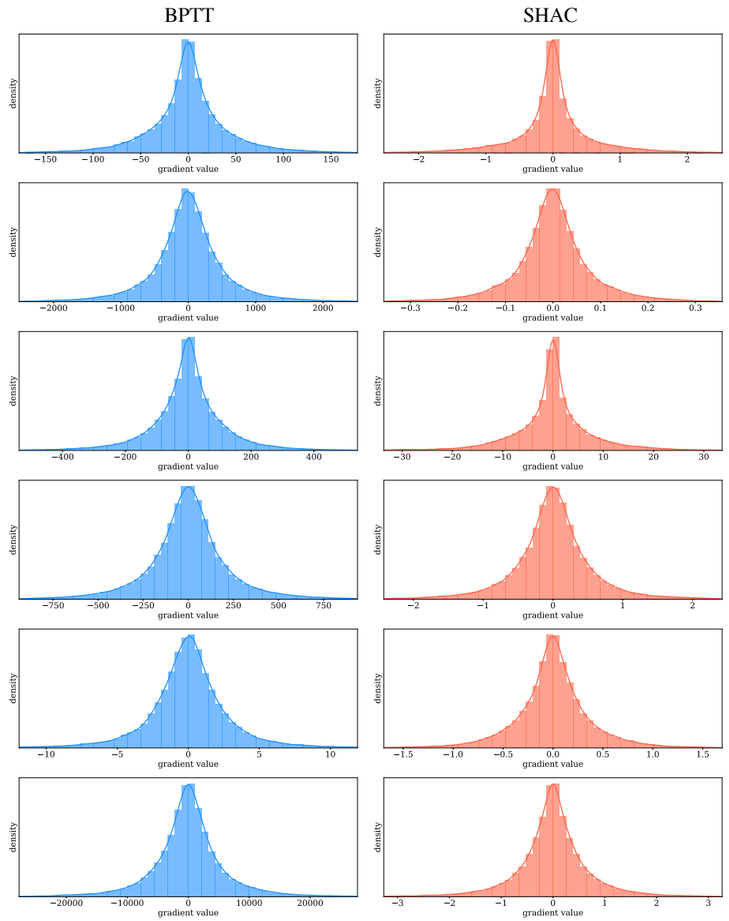

图 12：**训练过程中梯度分布的演变**。我们从 *Humanoid* 问题的训练过程中选取了六个不同回合的策略，并绘制了 BPTT（左）和 SHAC（右）的梯度值分布图。请注意图中的 x 轴刻度。图表显示，BPTT 计算出的梯度存在梯度爆炸问题，而我们方法计算出的解析梯度则更加稳定。

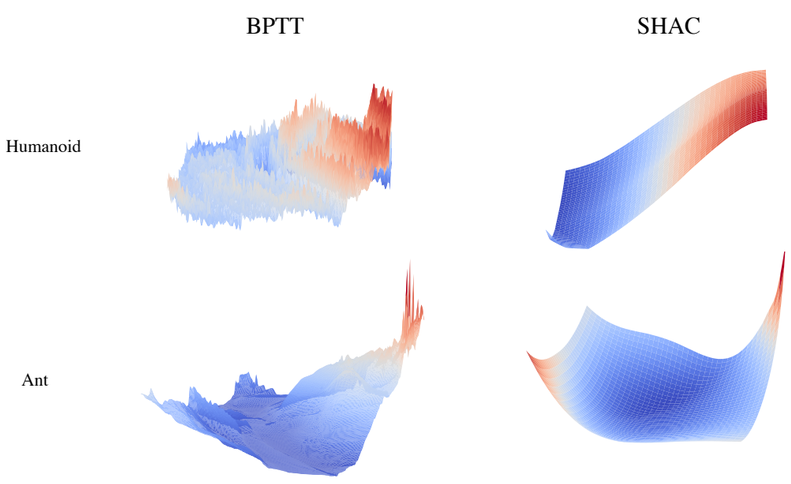

Figure 13: **Landscape surfaces comparison between BPTT and SHAC.** We study the landscapes for the *Humanoid* and *Ant* problems. For *Humanoid*, we choose a policy during the optimization and for *Ant* we choose the best policy from the learning algorithm. For each problem, we randomly select two directions in the policy parameter space and evaluate the policy variations along these two directions. The task horizon is $H = 1000$ for BPTT, and the short-horizon episode length for our method is $h = 32$. As we can see, longer optimization horizons lead to noisy loss landscape that are difficult to optimize, and the landscape of our method approximates the real landscape but is much smoother.

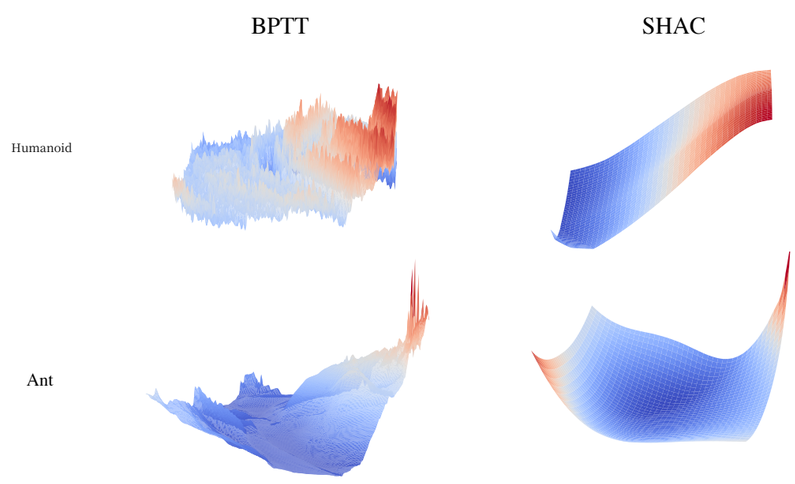

图 13：**BPTT 与 SHAC 的景观曲面比较**。我们研究了*Humanoid* 和*Ant* 问题的损失景观。对于*Humanoid*，我们在优化过程中选择一个策略；对于*Ant* ，我们选择学习算法中的最佳策略。对于每个问题，我们在策略参数空间中随机选择两个方向，并评估沿这两个方向的策略变化。BPTT 的任务时间范围为 $H = 1000$ ，而我们方法的短时间范围回合长度为 $h = 32$。正如我们所见，更长的优化时间范围会导致噪声较大的损失景观，难以优化；而我们方法的景观近似于真实景观，但更加平滑。